# From CI Telemetry to Operational Information Profiles: A Method for Android Instrumentation Testing in GitHub Actions

**Abstract**

Continuous integration (CI) platforms generate rich workflow telemetry, yet raw run, job, and step traces do not directly provide the kind of operational information needed for timely and defensible decision making. This gap is especially visible for Android instrumentation testing, where feedback latency is shaped not only by test execution, but also by setup, orchestration, environment instability, and heavy-tailed completion behavior. We present a telemetry-driven method for transforming CI workflow traces into interpretable operational profiles and decision-support outputs, instantiated on Android instrumentation testing in GitHub Actions. The study constructs operational information profiles for four common emulator-based testing styles: Community wrappers (Community), custom setups (Custom), Gradle Managed Devices (GMD), and third-party device farms (Third-Party). The method combines workflow evidence with run/job/step telemetry to construct a study-facing run×style dataset, represent elapsed time through two aligned timing layers, separate comparable execution regimes, and compare styles through four operational information dimensions: feedback timeliness, predictability, overhead placement, and deployment/outcome context. The goal is to support practitioners in choosing the style that best fits their operational needs and in identifying where their current style is most likely to benefit from improvement. On 8,845 executed run×style records, the method shows that the four styles form distinct operational information profiles rather than a single global ranking: Community yields the fastest typical completion but higher variability; GMD offers the strongest predictability-first and fast-entry profile; Third-Party carries the strongest exposure to very long runtimes; and Custom remains a sparse mixed case with relatively early entry but a much heavier completion tail. The styles also differ in usable-verdict rate, success among usable outcomes, and trigger context. We synthesize these results into an interpretable operational profile, a profile-derived decision-support guide, and a refreshable artifact companion that regenerates the same information products on later telemetry snapshots under the same analytical logic.

*Keywords:* CI telemetry, operational information profiles, GitHub Actions, Android instrumentation testing, workflow mining, workflow analytics, predictability, decision support

## 1. Introduction

Telemetry-rich digital infrastructures generate large volumes of workflow records, but volume alone does not guarantee usable information. Modern continuous integration (CI) platforms make rich workflow activity records available at scale [1, 2], yet workflow evaluation and adjustment are still often organized around coarse completion-time summaries [3]. This creates an information problem as much as a performance problem: raw workflow records are available, but the transformation from recorded workflow data to interpretable, decision-relevant operational knowledge remains weak. Android instrumentation testing provides a demanding empirical setting for this problem. These tests offer realistic end-to-end validation, but their observed feedback time is shaped not only by test execution itself, but also by emulator setup, environment preparation, orchestration, artifact handling, and environmental instability. Prior work identified recurring execution styles for running Android instrumentation tests in GitHub-based CI—Community wrappers, Custom setups, Gradle Managed Devices (GMD), and Third-Party device farms—and showed that these styles are widely used in practice [4]. What remains underdeveloped is a method for converting telemetry from these styles into structured operational information that supports comparison, interpretation, and later refresh as telemetry evolves.

A central challenge is that runtime alone is an incomplete information product. Two styles may appear similar on total completion time while differing substantially in stability, tail exposure, overhead placement, or the rate at which

runs produce usable outcomes. Observational comparisons are also shaped by attempt regime, trigger context, and workflow structure. Accordingly, practitioners need more than a single summary metric: they need an interpretable representation that explains how quickly feedback arrives, how stable that feedback is, where latency accumulates, and whether the outcomes are actionable in the deployment contexts where the workflow is actually used.

This paper addresses that need by presenting a *telemetry-driven method for transforming CI workflow traces into interpretable operational profiles*. The method is applied to Android instrumentation testing in GitHub Actions and empirically validated in that setting. It combines workflow evidence with run, job, and step telemetry to construct a study-facing run×style dataset, define aligned timing representations, separate comparable execution regimes, and generate interpretable decision-support outputs. In this sense, the contribution is not only a comparison of test execution styles, but also a generalizable information-processing pipeline for converting raw workflow traces into structured, interpretable, and refreshable decision support.

The method organizes this transformation through four complementary information dimensions. *Feedback timeliness* captures how quickly operational signals become available. *Predictability* captures stability and exposure to costly upper tails. *Overhead placement* explains where latency accumulates and therefore where intervention is most likely to matter. *Deployment/outcome context* extends the timing view by showing how styles are used across trigger regimes and how often they produce usable and successful outcomes. Together, these dimensions shift the analysis from runtime comparison to operational information design.

The paper makes four contributions. First, it introduces a telemetry-driven analytical design that constructs a study-facing run×style dataset from workflow, run, job, and step telemetry and represents elapsed time through two aligned layers: a broad job-level decomposition and a more precise step-level decomposition. Second, it defines a regime-aware comparison strategy that separates timing-focused analyses from outcome-context analyses and validates the main timing regime through a three-step validation procedure. Third, it introduces a workflow-shape fingerprint encoded as a signature hash and uses it as a downstream robustness layer for checking whether the main conclusions remain visible across structurally comparable workflow settings. Fourth, it demonstrates the method on a large snapshot of Android instrumentation testing in GitHub Actions and synthesizes the results into an interpretable operational profile, a profile-derived decision-support guide, and a refreshable artifact companion.

Empirically, the method shows that the four studied styles form distinct operational information profiles rather than a single universal ranking. The observed differences are visible not only in feedback timeliness, but also in predictability, overhead placement, and deployment/outcome context. These results support the paper's central claim: telemetry from Android instrumentation testing becomes more useful when it is converted into interpretable operational profiles rather than treated as runtime alone.

Conceptually, the proposed approach can be read as a simple input–process–output transformation from telemetry data to operational information, as illustrated in Figure 1. A raw GitHub Actions telemetry snapshot serves as the input. The process is the telemetry-driven method, which structures telemetry into a study-facing dataset, interprets it through the operational-information dimensions instantiated in this study as RQ1–RQ4, and synthesizes the resulting findings into interpretable outputs. The outputs are an operational profile and a profile-derived decision-support guide.

To support reproducibility and later refresh, we also provide a refreshable artifact companion that operationalizes the same method beyond the paper snapshot [5]. The artifact preserves the extraction and analysis logic and re-applies it to later telemetry snapshots so that the operational profile and decision-support outputs can be regenerated under the same analytical logic, without redefining the study design or introducing a new validation regime.

The remainder of the paper is organized as follows. Section 2 reviews the background and related work. Section 3 presents the method, including dataset construction, timing representation, comparison setup, statistical analysis, validation, and artifact operationalization. Section 4 reports the empirical analysis. Section 5 synthesizes the results into operational profiles and profile-derived decision-support outputs. Section 6 discusses the study's implications. Section 7 reviews threats to validity, and Section 8 concludes.

## 2. Background and Related Work

### *2.1. Operational telemetry as an information problem*

Workflow platforms increasingly generate detailed operational traces, but the availability of traces does not by itself yield usable information. Information-processing research has long shown that information use is shaped by

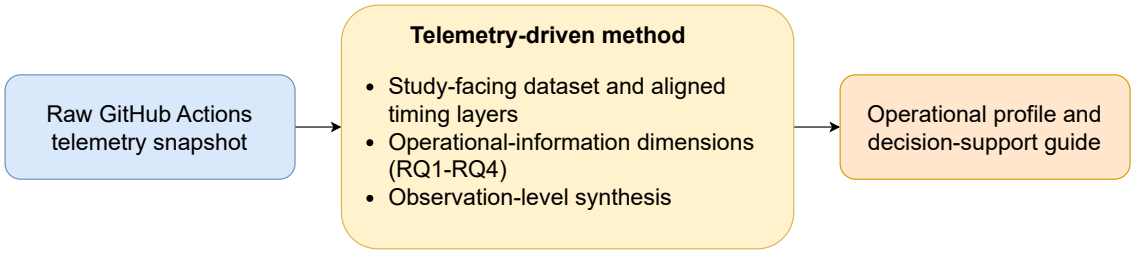


Figure 1: System-level view of the proposed method for transforming GitHub Actions telemetry into operational information.

task context, information pathways, and the relation between information needs and the forms in which signals are made available [6–9]. In this perspective, CI telemetry is not only a technical by-product of software execution; it is an operational information environment in which raw records must be translated into representations that decision makers can interpret and act upon.

This framing matters because runtime-only summaries compress away information that practitioners need in order to diagnose and compare workflow behavior. A useful operational representation must preserve distinctions among timeliness, stability, bottleneck placement, and outcome actionability rather than collapsing them into one number. Our work therefore treats CI telemetry as a source of structured operational information and asks how that information can be transformed into interpretable profiles and refreshable decision-support outputs.

### *2.2. Android instrumentation testing as the empirical setting*

Android instrumentation tests execute end-to-end scenarios on emulators or physical devices and are widely used to validate application behavior under realistic runtime conditions [10–12]. Because they run on-device, they inherit environment- and timing-sensitive behavior that is less common in local JVM tests. In CI, instrumentation testing is operationally challenging due to device provisioning, dependence on SDK and emulator tooling, and nondeterministic runtime conditions that can amplify flakiness and tail latency [13–16].

Our earlier baseline study contributed a candidate-project identification procedure and an execution-style taxonomy for Android instrumentation testing in GitHub-based CI [4]. The present paper builds on that foundation, but shifts the focus from style identification to the construction of operational information products that make style behavior interpretable under different objectives.

### *2.3. Execution substrates and observable workflow differences*

Modern mobile CI pipelines realize instrumentation tests through distinct execution substrates: hosted or self-hosted emulators, build-managed devices, and cloud device farms. Gradle Managed Devices (GMD) formalize a build-managed approach in which the Android Gradle Plugin manages device creation, deployment, and teardown around test execution [17]. Cloud device-farm offerings such as Firebase Test Lab expose remote execution workflows that add provider-side orchestration and result handling [18, 19]. These substrates differ not only in typical runtime, but also in where overhead is likely to accumulate and how styles are deployed across workflow triggers.

### *2.4. From traces to interpretable operational representations*

CI platforms expose run-, job-, and step-level telemetry that supports large-scale observation of timing, outcomes, attempts, and workflow structure [1, 2]. Yet observational use of these data remains vulnerable to infrastructure effects, queueing, reruns, and workflow configuration differences [3, 20, 21]. Accordingly, telemetry becomes decision-relevant only when it is organized into comparable and interpretable representations.

Our method addresses this by combining two ideas. First, it constructs a study-facing run×style dataset from workflow evidence and telemetry rather than relying only on repository- local logging conventions. Second, it represents elapsed time through two aligned timing layers, allowing broad comparison and more precise step-centered interpretation within the same overall run duration. This design treats traces as raw material for information abstraction rather than as self-explanatory evidence.

### 2.5. *Interpretability and decision support*

A growing body of information-science research argues that analytics are most valuable when their outputs remain interpretable to human decision makers. Work in IPM on auditing black-box decision support systems and on explainable AI has emphasized the importance of transparent reasoning, trust, and human decision performance in data-driven environments [22, 23]. That concern aligns with the present study, where the goal is not merely to rank styles, but to produce operational profiles and objective- conditioned guidance that remain legible to practitioners.

For this reason, the paper favors interpretable profile synthesis over opaque recommendation models. We use descriptive and comparative evidence to generate operational profiles and a profile-derived guide rather than training a black-box style selector. The aim is to preserve inspectability, make trade-offs visible, and keep the decision logic close to the underlying telemetry evidence.

### 2.6. *Workflow shape, comparability, and robustness*

Observational comparisons can be distorted when the factor of interest is entangled with differences in execution shape. Information-processing research has long noted that task complexity and context change how information is sought and used [6, 7]. In telemetry-rich workflow environments, an analogous problem arises when style comparisons are mixed across structurally dissimilar executions.

We address this through a workflow-shape fingerprint derived from runner family and coarse executed-workload characteristics. The fingerprint is not used as a primary controller, but as a robustness layer for checking whether the main conclusions remain visible within more comparable workflow settings. This supports a more context-sensitive reading of telemetry and helps separate style-level patterns from workflow-shape effects.

### 2.7. *Refreshable analytical artifacts as information products*

The practical value of telemetry analysis depends not only on a single static study, but also on whether the same analytical logic can be re-applied as traces evolve. Software citation and artifact studies have emphasized discoverability, reuse, and maintenance of research artifacts [24–26]. In our setting, the artifact layer is important because it preserves the transformation from raw telemetry to operational profiles and decision-support outputs. The contribution is therefore not just a one-time analysis, but a refreshable analytical companion that can regenerate the same information products on later snapshots under the same method.

### 2.8. *Positioning of the present study*

Taken together, the prior literature suggests three open needs. First, CI telemetry needs to be framed as an information resource rather than only a performance trace. Second, operational comparison requires interpretable abstractions that preserve trade-offs rather than runtime alone. Third, analytical outputs need refreshable forms if they are to remain useful in evolving workflow ecosystems. The present paper addresses these needs by proposing and validating a telemetry-driven method for transforming CI workflow traces into interpretable operational profiles and decision-support outputs, with Android instrumentation testing in GitHub Actions as the empirical setting.

## 3. Methodology

This study develops and validates a telemetry-driven method for transforming CI workflow traces into interpretable operational information. The empirical setting is Android instrumentation testing in GitHub Actions, and the analytical unit is the **run×style** observation. The method builds on the candidate-project identification procedure and the instrumentation-testing execution-style taxonomy introduced in [4]. Following that taxonomy, we focus on four emulator-based execution styles: **Community**[1], **Custom**, **GMD**, and **Third-Party**. We exclude `Real-Devices` because they are sparse in the current snapshot and reflect operational assumptions that differ qualitatively from emulator-based CI execution.

[1] We retain the taxonomy label *Community* from [4]. In this GitHub Actions-based study, it operationally refers to the GitHub Actions instantiation of that style.

Figure 2: Study-facing data construction pipeline and datasets by stage (snapshot: Mar. 2026). The pipeline is shown after excluding `Real-Devices` from the four-style analytical scope.

Operationally, emulator-testing style performance is not a single property reducible to runtime alone. For workflow owners and decision makers, a usable information representation must preserve at least four aspects of style behavior: how quickly feedback becomes available, how stable that feedback is, where observable delay accumulates, and whether the resulting run outcomes are actionable in the CI contexts where the style is actually deployed [6, 7, 22, 23]. In this study, these four operational-information needs are instantiated as RQ1–RQ4: feedback timeliness, predictability, overhead placement, and deployment/outcome context.

### *3.1. From workflow telemetry to a study-facing dataset*

The method has two linked parts. The first is an *extraction pipeline* that collects historical GitHub Actions telemetry. The second is a *representation step* that transforms those records into a study-facing dataset for comparative and interpretive analysis. This design treats workflow telemetry not as self-explanatory evidence, but as raw observational material that must be structured before it can support interpretation and decision making.

The extraction side combines the workflow-screening and style-detection heuristics introduced in [4] with historical run/job/step telemetry from the GitHub Actions REST API [1, 2]. We rely on API-exposed telemetry rather than raw logs because it provides a scalable and structured historical view across repositories, while raw logs and artifacts may be missing or truncated under repository retention policies [27]. The pipeline proceeds in three stages, summarized in Fig. 2.

*Inputs and scope.* We begin from **481 candidate repositories** identified by the baseline procedure [4]. The study-facing scope is then restricted to the four emulator-based styles (**Community**, **Custom**, **GMD**, **Third-Party**).

*Stage 1: Constructing the workflow inventory.* Stage 1 constructs a broad workflow inventory by scanning GitHub workflow configuration files together with the repository-local scripts and references invoked by those workflows. The purpose of this stage is to identify instrumentation-related workflow evidence and label the execution-style signals needed for downstream extraction. In the current snapshot, this stage yields **1,637 workflow records** from **306 repositories**. This stage defines the screened workflow universe rather than the final analytical set.

*Stage 2: Constructing the run inventory.* Stage 2 enumerates historical runs for the Stage 1 workflow inventory and constructs the study's run inventory from run- and job-level telemetry. In the current snapshot, this stage yields **22,338 runs** from **178 run-observed instrumentation-capable workflows** across **140 repositories**. Because a single workflow run may expose more than one execution style, Stage 2 also materializes a derived **run×style** view. After restricting this view to the four study styles and excluding `Real-Devices`, the inventory contains **22,407 run×style records**. This Stage 2 output defines the run-level inventory carried forward to Stage 3, where instrumentation-executed records are identified and step-level telemetry is extracted for the final study dataset.

*Stage 3: Constructing the executed run×style dataset.* Stage 3 starts from the Stage 2 run×style inventory and retains only records where instrumentation testing actually executed, using style-specific instrumentation-job evidence. Operationally, this stage keeps rows with executed instrumentation-job support and, for that instrumentation-test-executed subset, pulls the corresponding **step telemetry**. Stage 3 then derives the study's precise Layer 2 timing components, records the selected cutpoints and step-level breakdown output, and extracts the step-based interpretation fields required later in the study, including the grouping dimensions used in the overhead-focused analyses. After restricting to the four study styles, Stage 3 yields **8,845 instrumentation-test-executed run×style records**. These records form the extraction output used for final study-dataset construction.

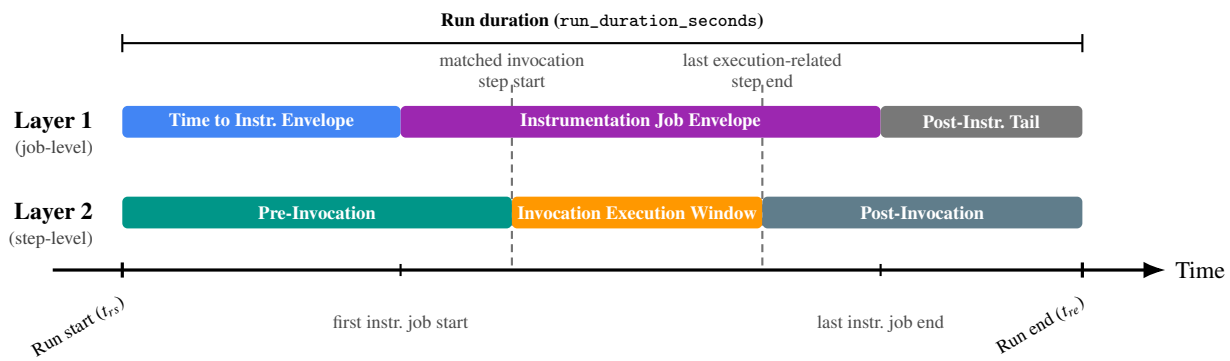


Figure 3: Study-wide run-timeline decomposition. Layer 1 provides the broad job-level view, and Layer 2 provides the more precise step-level view. Both layers span the same run duration.

### *3.2. Timeline representation*

Figure 3 defines the study's central analytical representation. The method uses two aligned timing layers over the same observed run duration: a broad job-level layer (**Layer 1**) and a more precise step-level layer (**Layer 2**). Layer 1 supports durable high-level comparison across styles, while Layer 2 refines the same elapsed interval when step telemetry is directly available.

*Global run cut points.* We define **run start** ($t_{rs}$) and **run end** ($t_{re}$) from workflow-run telemetry. These cut points define **RunDuration** as:

$$\text{RunDuration} = t_{re} - t_{rs}.$$

*Layer 1: broad job-level representation.* Layer 1 is defined by four cut points: run start, first instrumentation-related job start, last instrumentation-related job end, and run end. It yields three broad components:

- **Time to Instrumentation Envelope**,
- **Instrumentation Job Envelope**,
- **Post-Instrumentation Tail**.

These three components sum to **RunDuration**.

*Layer 2: invocation-centered step-level representation.* Layer 2 is defined by four cut points: run start, matched invocation step start, last execution-related step end, and run end. It yields three more precise components:

- **Pre-Invocation**,
- **Invocation Execution Window**,
- **Post-Invocation**.

These also sum to **RunDuration**.

This cutpoint-based design is intentional. The goal is not to label every step with a perfect semantic category, but to construct an additive, interpretable decomposition of elapsed time that remains comparable across runs and styles. Defining the middle segment as an elapsed interval preserves a clean wall-clock decomposition of the run. By contrast, summing only selected step durations would leave intervening time harder to interpret and weaken validation against total run duration.

### *3.3. Comparison setup, robustness, and coverage*

This subsection specifies how the study organizes comparison, coverage, and robustness for the main analyses.

*Main comparison factor.* The study compares four emulator-based execution styles: **Community**, **Custom**, **GMD**, and **Third-Party**.

*Core and contextual variables.* The seven timeline components introduced above constitute the core analytical representation of the study: one overall duration, three Layer 1 components, and three Layer 2 components. Around them, the method uses a smaller set of context and interpretation variables: attempt status, usable verdict, trigger event, step activity group, execution role, and workflow-shape fingerprint. These supporting variables are not treated as co-equal outcome measures. Instead, they help define the main comparison regime, interpret overhead placement, and support downstream robustness checks.

*Workflow-shape fingerprint.* After construction of the executed instrumentation-test run×style dataset, a workflow-shape fingerprint is derived to group structurally similar runs for later robustness checks. It is constructed from three coarse executed-workload dimensions: runner OS family, executed job-count bucket, and executed step-count bucket. In the current study, these dimensions are defined from the *executed* workflow structure rather than the coded workflow specification. The resulting grouping is encoded as a stable signature hash and used only as a downstream robustness device, not as a primary controller [28]. Its role is to check whether the main findings remain visible within more comparable workflow settings.

*Role of the robustness layer.* The fingerprint is used after the main pooled analyses have been completed. Tier 1 uses eligible exact signatures, where eligibility means satisfying the study's minimum-support and minimum usable cross-style-overlap requirements. Tier 2 then defines a paired coarsened family around each eligible exact signature by holding the runner-OS bucket fixed and broadening the executed job-count and executed step-count buckets to adjacent categories. Robustness checks are then conducted for the selected key observations in this study. The exact Tier 1 eligibility criteria and Tier 2 coarsened-family construction rule are provided in 9.1, and the resulting eligible-signature coverage summary is reported in Table 9.

*Base timing regime.* The main timing analyses use the **Base** regime: instrumentation-test-executed runs that are first attempts and that produce a usable run-level verdict (`success` or `failure`). This regime is used for the timing-focused research questions.

*Final study dataset and coverage.* The final study dataset is defined at the **run×style** level. It combines (i) run-level metadata and broad timing components, (ii) step-derived timing components where directly observable, and (iii) context fields used for interpretation, comparability, and robustness analysis. In the current snapshot, the four-style dataset contains **8,845** instrumentation-test-executed run×style records. Applying the first-attempt and usable-verdict controllers yields the Base timing regime with **7,744** records. Coverage is effectively complete for **Layer 1**, because its timing components are derived from run/job telemetry. By contrast, **Layer 2** is directly observable for **5,057** Base records (**65.30%**), reflecting the more limited availability of usable step-level telemetry in public GitHub Actions repositories.

*Comparison logic.* Because the study is observational, the analyses use a controlled comparison regime and a separate robustness layer. The controlled regime improves comparability for the main timing analyses. The robustness layer then checks whether the same conclusions remain visible within more structurally similar workflow settings.

### *3.4. Statistical analysis*

Timing outcomes are often heavy-tailed and non-normal. Accordingly, the study uses non-parametric procedures and emphasizes effect sizes as indicators of practical impact rather than relying on parametric assumptions [29, 30].

*Timing comparisons.* For overall multi-style timing comparisons, we use Kruskal–Wallis tests and report $\epsilon^2$ as an effect size [30, 31]. For targeted pairwise interpretation, we use Mann–Whitney tests [32] with Holm correction [33] and report Cliff's $\delta$ [34]. For Cliff's $\delta$, we interpret effect sizes using the common negligible/small/medium/large thresholds of 0.147, 0.330, and 0.474 [35, 36].

Table 1: Summary of Step 1 automated rule-based validation. Values report the *average scope-based in-scope accuracy across rules* within each validation category. For Layer 1 and broad study-facing rules, Overall rates are measured on the full dataset ($n$ = 8,845 run×style records) and Base rates on instrumentation-executed first-attempt usable-verdict records ($n$ = 7,744). For Layer 2 and step-telemetry-dependent rules, Overall rates are measured on `Step_telemetry=True` ($n$ = 5,747) and Base rates on `Step_telemetry=True` within Base ($n$ = 5,057).

| Category | What was checked | Overall | Base |
|---|---|---|---|
| Structural integrity | Key uniqueness, required identifiers, style-scope validity, and expected study flags | 100.00% | 100.00% |
| Temporal consistency | Run-boundary ordering, execution-window ordering, cutpoint ordering, and window-within-run consistency | 100.00% | 100.00% |
| Layer 1 equation validity | Whether run duration equals the sum of the three Layer 1 components | 98.45% | 99.96% |
| Layer 2 recomputation validity | Whether the Layer 2 components are non-negative, bounded by run duration, and consistent with cutpoints and run boundaries | 100.00% | 100.00% |
| Source traceability | Whether selected invocation and execution-end cutpoints uniquely and correctly match the step-level source records | 99.75% | 99.79% |
| Auxiliary consistency | Whether the derived workflow-shape, repeated-style, and invocation-window support fields are present and internally coherent | 100.00% | 100.00% |

*Predictability and tail summaries.* To characterize predictability and tail exposure, we report IQR, IQR/median, P95, and P95/median where appropriate. These summaries are used alongside the non-parametric comparisons to separate typical performance from upper-tail risk [30, 37].

*Categorical outcomes.* For categorical outcomes, including verdict and trigger-based comparisons, we use contingency-table analysis with $\chi^2$ tests and report Cramer's $V$ as an effect size [38]. Where useful, we report confidence intervals for key proportions using established single-proportion methods [39].

### *3.5. Validation procedure*

Before turning to the RQ-specific analyses, we validate the method through three complementary steps addressing representation integrity, source-grounded correctness, and context robustness. The goal is to confirm that the staged pipeline identifies the intended executions, that the timing representation behaves coherently, that the main comparison regime is justified, and that the resulting profile picture is not driven by structurally unusual workflow settings.

*Step 1: Automated representation validation.* The first step is an automated review of the analysis dataset. It verifies that the staged pipeline produces structurally valid, temporally coherent, and arithmetically consistent run×style observations before the RQ-specific analyses begin.

The automated validation comprises 54 rules spanning six aspects of dataset integrity: structural integrity, temporal consistency, Layer 1 equation validity, Layer 2 recomputation validity, source traceability, and auxiliary consistency. We report these checks for both the full dataset and *Base*, the study's main timing regime. Table 1 summarizes the category-level results.

As shown in Table 1, the automated validation indicates very strong internal consistency. Structural integrity, temporal consistency, Layer 2 recomputation validity, and auxiliary consistency are perfect in both views. The only non-perfect category-level results arise in Layer 1 equation validity and source traceability, both of which remain near-perfect. These small residual deviations are limited to a few in-scope missing cases in the Layer 1 equation check and a small number of cutpoint-selection and source-match cases rather than broader representation errors.

The checks are evaluated only where they should be available under the study's own measurement design, so missing values within scope contribute to the observed failure burden rather than being excluded. The rule definitions, scope logic, equation checks, and detailed rule-level results are reported in Appendix 9.2, especially Appendix 9.2.1 and Table 10. Taken together, Step 1 shows that the study's timing representation rests on a highly coherent measurement foundation.

Table 2: Summary of Step 2 stratified manual audit results. Accuracy is reported on an applicable-record basis, computed as *correct* / (*correct* + *mismatch*), excluding non-auditable cases.

| **Checked field** | **Full sample** | **Base** |
|---|---|---|
| In-scope inclusion | 100.00% | 100.00% |
| Instrumentation executed | 100.00% | 100.00% |
| Style assignment | 100.00% | 100.00% |
| Layer 1 boundaries | 100.00% | 100.00% |
| Layer 2 invocation anchor | 96.39% | 100.00% |
| Layer 2 execution-end anchor | 96.39% | 100.00% |
| Step activity group | 100.00% | 100.00% |
| Execution role | 100.00% | 100.00% |
| Overhead phase | 100.00% | 100.00% |
| Attempt field | 100.00% | 100.00% |
| Run-level verdict/controller field | 100.00% | 100.00% |

*Step 2: Stratified source audit.* The second step is a stratified manual audit of sampled run×style records against their source workflow-run evidence. Whereas Step 1 validates internal consistency at scale, Step 2 validates whether the constructed observations remain faithful to the underlying execution evidence.

The audited sample contains 124 records drawn from Base and from targeted supplementary strata designed to stress more fragile cases. The sample covers all four study styles. For each sampled record, we manually check in-scope inclusion, instrumentation execution, style assignment, auditable Layer 1 and Layer 2 anchors, derived Layer 2 grouping labels, and the controller-facing fields used in the study.

The manual audit shows very strong agreement with the constructed dataset. In the full sampled set, all reviewed records are confirmed to be in scope, instrumentation-executed, correctly styled, and correct on the controller-facing fields. Layer 1 boundaries are correct for all auditable cases, and the Layer 2 grouping labels are correct for all auditable cases. The only residual disagreements are limited to three non-Base records affecting the Layer 2 anchor checks. In Base, the sampled manual audit is fully clean. Table 2 summarizes the field-level results.

These residual disagreements are confined to non-Base records whose audit-sheet observability state does not fully align with the current final dataset revision. Thus, the detected mismatches reflect a narrow observability-state alignment issue rather than broader errors in style detection, controller assignment, or grouping construction. Taken together, Step 2 provides strong source-grounded evidence that the constructed records and interpretation-facing fields are highly accurate.

*Step 3: Regime and context robustness.* The third step checks whether the study's main timing-profile picture remains stable under the comparison and robustness design defined in Section 3.3. It has two purposes: first, to assess whether choosing Base as the main timing regime materially changes the style-level picture relative to adjacent controller regimes; and second, to assess whether the workflow-shape layer provides usable within-shape settings for validating key findings.

Table 11 summarizes the validation-level outcomes. For controller-regime robustness, the full executed four-style view and the first-attempt view each preserve the Base ordering exactly for six of the seven timing measures, with one additional measure only partially preserved. By contrast, the rerun usable-verdict regime does not preserve any of the seven orderings exactly. This supports retaining Base as the main timing-comparison regime and treating reruns as a distinct adjacent condition.

For workflow-shape robustness, the current snapshot yields 20 candidate exact signatures, of which only two satisfy the Tier 1 eligibility rule for deeper follow-up. Neither includes all four styles, so exact four-style ordering checks are not possible in the current snapshot. Accordingly, the workflow-shape follow-up is interpreted only in partial-order terms over the styles present within each eligible setting. Under this interpretation, the exact-signature and paired-family checks remain informative, but they serve as constrained within-shape robustness support rather than as a standalone replacement for the pooled Base comparison.

Overall, Step 3 supports two conclusions: Base remains a justified main comparison regime for the timing-focused analyses, and the workflow-shape fingerprint provides a meaningful but limited context-robustness layer.

Table 3: Median timing profile by style under the Base timing regime for RQ1 timeliness analysis.

| | | Layer 1 | | | Layer 2 | | |
|---|---|---|---|---|---|---|---|
| **Style** | **Run dur.** | **Time to instr.** | **Instr. env.** | **Post-instr.** | **Pre-invoc.** | **Exec. window** | **Post-invoc.** |
| Community | **883** | 4 | **624** | 7 | 187 | **397** | 75 |
| GMD | 1212 | **3** | 1208 | **1** | **79.5** | 1076 | **4** |
| Third-Party | 1864.5 | 4 | 1717.5 | **1** | 561 | 1881.5 | 5 |
| Custom | 1817 | **3** | 1609.5 | **1** | 131.5 | 726 | 942.5 |

*Note:* Values are medians in seconds. Layer 2 measures are based on the step-observable Base subset. Bold indicates the smallest median in each timing measure.

## 4. Empirical Analysis and Results

In this section, we present our empirical findings across the research questions. For each RQ, we describe the motivation, explain the analytical approach, and report the key findings.

### *4.1. RQ1: How do styles differ in developer feedback speed?*

RQ1 examines how the four emulator-testing styles differ in *feedback timeliness* under the controlled Base regime (**n = 7,744**). It compares styles on overall run duration, the three **Layer 1** timing components (Time to Instrumentation Envelope, Instrumentation Job Envelope, and Post-Instrumentation Tail), and the three **Layer 2** timing components (Pre-Invocation, Invocation Execution Window, and Post-Invocation) where directly observable. Rather than treating speed as a single runtime value, we interpret timeliness through the study's two-layer timing representation: Layer 1 supports broad high-level comparison across styles, while Layer 2 supports more precise invocation-centered interpretation on the step-observable Base subset (**n = 5,057**). This allows us to distinguish between overall completion speed, speed of reaching the instrumentation path, speed within the main execution window, and the extent to which a style carries a residual completion tail. Table 3 summarizes the central timing profile by style.

*Obs. 1.1 (Community provides the fastest completion-oriented timeliness profile).* Community forms the clearest *fast-completion* profile in the current Base regime. As shown in Table 3, it has the shortest median overall run duration (**883s**), ahead of GMD (**1212s**), Third-Party (**1864.5s**), and Custom (**1817s**). These differences are statistically significant relative to GMD ($p = 1.90 \times 10^{-11}$, $\delta = -0.244$, small), Third-Party ($p = 1.71 \times 10^{-54}$, $\delta = -0.406$, small-to-medium), and Custom ($p = 4.27 \times 10^{-3}$, $\delta = -0.284$, small). The same reading is visible in the central timing segments: Community also has the smallest instrumentation envelope and the smallest invocation-execution window. In profile terms, Community is the style most associated with faster overall feedback delivery. This main result is also supported, though qualifiedly, by the study's two-tier workflow-shape robustness check, which applies the same comparison logic and statistical tests within more comparable workflow settings. In this follow-up, Tier 1 provides qualified support, mainly because the Community-versus-GMD completion contrast weakens in the exact-signature setting, whereas Tier 2 preserves the same completion-oriented interpretation more clearly. Detailed Tier 1 and Tier 2 support is reported in Tables 12, 13, 14, 15, and 16.

*Obs. 1.2 (GMD reaches the instrumentation path earliest, but is not the fastest to complete).* GMD forms the clearest *fast-entry* profile rather than the fastest completion profile. In Layer 1, its median time to instrumentation envelope is **3s**, lower than Community and Third-Party (**4s**) and tied with Custom; relative to Community, this difference is highly significant ($p = 6.66 \times 10^{-29}$, $\delta = -0.399$, small-to-medium), while Custom is effectively tied with GMD ($p = 0.722$). The same pattern appears more clearly in Layer 2 pre-invocation, where GMD has the smallest median (**79.5s**); the contrast against Third-Party is especially strong ($p = 6.30 \times 10^{-38}$, $\delta = -0.949$, very large). However, GMD does not convert this earlier entry into the fastest end-to-end completion. It remains slower than Community in both the invocation-execution window ($p = 3.03 \times 10^{-21}$, $\delta = 0.479$, large) and overall run duration. Thus, GMD is best interpreted as a style that reaches the instrumentation path earlier, but whose main execution path remains longer than Community's. This main result is also supported, though qualifiedly, by the study's two-tier workflow-shape robustness check, which applies the same comparison logic and statistical tests within more comparable workflow

settings. In this follow-up, the fast-entry interpretation is preserved most clearly relative to Third-Party, while the Community-versus-GMD entry contrast remains weak in both tiers. Detailed Tier 1 and Tier 2 support is reported in Tables 12, 13, 14, 15, and 16.

*Obs. 1.3 (Third-Party shows the slowest sustained-execution timeliness profile).* Third-Party forms the clearest *slow-path* profile in RQ1. Table 3 shows that it has the longest median overall run duration, the longest pre-invocation segment, and the longest invocation-execution window among the four styles. Relative to Community, it is markedly slower in overall duration ($p = 1.71 \times 10^{-54}$, $\delta = 0.406$, small-to-medium) and even more clearly slower in the invocation-execution window ($p = 1.42 \times 10^{-28}$, $\delta = 0.607$, large). It also shows a very large disadvantage relative to GMD in pre-invocation time ($p = 6.30 \times 10^{-38}$, $\delta = 0.949$, very large). Compared with Custom, Third-Party is similar in overall duration ($p = 0.149$), but it remains slower in the observable execution path, especially for pre-invocation ($p = 4.01 \times 10^{-9}$, $\delta = 0.700$, large) and invocation-execution ($p = 2.57 \times 10^{-7}$, $\delta = 0.613$, large). In timeliness terms, Third-Party is therefore best understood not simply as "slower overall," but as the style whose delays accumulate both before invocation and during the main execution window. This main result is also strongly supported by the study's two-tier workflow-shape robustness check, which applies the same comparison logic and statistical tests within more comparable workflow settings. In this follow-up, both Tier 1 and Tier 2 preserve the slow sustained-execution interpretation, with strong support for the key contrasts against Community and GMD. Detailed Tier 1 and Tier 2 support is reported in Tables 12, 13, 14, 15, and 16.

*Obs. 1.4 (Custom remains a mixed timeliness case shaped by a heavy completion tail).* Custom contributes a more cautious and mixed timeliness profile. Its entry behavior is competitive: in Layer 1, its median time to instrumentation envelope matches GMD and improves on Community (**3s** vs. **4s**; $p = 7.01 \times 10^{-3}$, $\delta = -0.264$, small), while remaining directionally faster than Third-Party ($p = 0.119$). However, this early entry does not lead to faster completion. Custom is significantly slower than Community ($p = 4.27 \times 10^{-3}$, $\delta = 0.284$, small) and GMD ($p = 4.97 \times 10^{-7}$, $\delta = 0.530$, medium) on overall run duration, and its instrumentation envelope is also much longer than Community's and GMD's. In Layer 2, its main distinguishing feature is the very long post-invocation tail (**942.5s**), far above Community, GMD, and Third-Party, with especially strong differences relative to GMD ($p = 1.91 \times 10^{-12}$, $\delta = 0.812$, large) and Third-Party ($p = 2.83 \times 10^{-7}$, $\delta = 0.602$, medium-to-large). Accordingly, Custom is best read as a mixed-speed style whose competitive entry is offset by a much heavier completion tail.

*Obs. 1.5 (Community's speed advantage is concentrated in the main execution path, not uniformly across all segments).* Although Community is the fastest overall style, its timing advantage is not uniform across the entire run. Its main strength lies in the execution-centered middle of the timeline, where it has the smallest instrumentation envelope and the smallest invocation-execution window. That advantage is followed by a somewhat longer residual tail than GMD and Third-Party. In Layer 1 post-instrumentation tail, Community records a median of **7s**, compared with **1s** for GMD, Third-Party, and Custom; these differences are highly significant relative to GMD ($p = 5.56 \times 10^{-52}$, $\delta = 0.535$, large), clear relative to Third-Party ($p = 8.31 \times 10^{-39}$, $\delta = 0.330$, medium), and still significant relative to Custom ($p = 1.25 \times 10^{-5}$, $\delta = 0.422$, small-to-medium). The same pattern appears more clearly in Layer 2 post-invocation, where Community remains above GMD and Third-Party, while Custom shows an even heavier tail. This means that Community should not be interpreted as uniformly faster in every segment; rather, its timeliness advantage is concentrated in the core execution path and coexists with a modest residual continuation phase.

**RQ1 Summary (Timeliness profile).** The four styles exhibit distinct feedback-timeliness profiles under the controlled usable-verdict Base regime. Community provides the fastest completion-oriented profile. GMD provides the clearest fast-entry profile, but not the fastest end-to-end completion. Third-Party shows the slowest sustained-execution profile, with delays accumulating both before invocation and during the main execution window. Custom remains a mixed case whose comparatively competitive entry is offset by a much heavier completion tail. Taken together, these results show that feedback timeliness is not a single ranking, but a set of distinct operational timing profiles.

Table 4: Predictability profile by style under the Base timing regime for RQ2.

| Style | Norm. abs. dev. (run dur.) | Norm. abs. dev. (Instr. env.) | Norm. abs. dev. (L2 exec. window) | P95 run dur. (s) | P95 / median (run dur.) | Profile reading |
|---|---|---|---|---|---|---|
| Community | 0.506 | 0.511 | 0.514 | 13,909.0 | 15.75 | Fast but variable |
| GMD | 0.061 | 0.060 | 0.045 | 1,483.4 | 1.22 | Tightest / most predictable |
| Third-Party | 0.256 | 0.233 | 0.168 | 34.6M | 18,558.90 | Strongest absolute tail risk |
| Custom | 0.231 | 0.283 | 0.680 | 3,247.5 | 1.79 | Mixed, interpret cautiously |

*Note:* Base is defined as instrumentation-executed first-attempt run×style records with usable verdicts (success or failure), yielding $n = 7{,}744$ observations. Normalized absolute deviation is computed within style as $|x - \tilde{x}|/\tilde{x}$, where $\tilde{x}$ is the style-specific median for that measure. Layer 2 values are computed on the step-observable Base subset. For readability, the Third-Party P95 run duration is shown as 34.6M seconds.

### *4.2. RQ2: Which styles are most predictable, and which expose developers to the greatest tail-latency risk?*

RQ2 examines how predictable the four emulator-testing styles are under the controlled Base regime (**n = 7,744**). It evaluates predictability for run duration and the Layer 1 and Layer 2 timing components using the variability and tail summaries defined in Section 3.4. It then uses selected fingerprint-qualified workflow families as a downstream validation layer to check whether key predictability conclusions remain visible inside structurally comparable groups. Here, predictability is treated as an operational information dimension rather than as a single variability statistic. The focus is on whether a style remains close to its own typical behavior, how strongly it exposes developers to upper-tail delay, and whether that instability arises broadly across the run or more specifically within the main execution path. Table 4 summarizes the central predictability profile by style.

*Obs. 2.1 (GMD provides the strongest predictability-first profile).* GMD forms the clearest *stability-first* profile in the current usable-verdict Base. As shown in Table 4, it is consistently tighter than the other styles on the main completion-oriented measures. Its median normalized absolute deviation is **0.061** for overall run duration, compared with **0.506** for Community, **0.231** for Custom, and **0.256** for Third-Party. These contrasts are highly significant, with a large practical difference relative to Community ($p = 1.81 \times 10^{-100}$, $\delta = -0.772$, large) and clearly meaningful differences relative to Custom ($p = 2.15 \times 10^{-10}$, $\delta = -0.669$, medium-to-large) and Third-Party ($p = 7.95 \times 10^{-48}$, $\delta = -0.634$, medium-to-large). The same pattern holds for the Layer 1 instrumentation-job envelope and the Layer 2 invocation-execution window, where GMD again records the smallest normalized deviations. In profile terms, GMD is the style most associated with repeatable timing behavior rather than with raw completion advantage. This main result is also strongly supported by the study's two-tier workflow-shape robustness check, which applies the same comparison logic and statistical tests within more comparable workflow settings. In this follow-up, both Tier 1 and Tier 2 preserve GMD as the tightest style on the main completion-oriented predictability measures, with especially strong separation from Community and clear separation from Third-Party. Detailed Tier 1 and Tier 2 support is reported in Tables 12, 13, 14, 15, and 16.

*Obs. 2.2 (Community combines faster typical performance with much weaker predictability).* Community forms the clearest *speed–predictability tradeoff* in the study. In RQ1 it often provides the fastest typical completion, but under predictability-oriented measures it shows much heavier variation than GMD and noticeably heavier variation than the more stable profiles overall. As shown in Table 4, its median normalized deviation reaches **0.506** for run duration, **0.511** for the instrumentation-job envelope, and **0.514** for the invocation-execution window. Relative to GMD, these contrasts are consistently statistically significant and practically large, including run duration ($p = 1.81 \times 10^{-100}$, $\delta = 0.772$, large). Even relative to Custom, Community is less stable on broad completion-oriented measures such as run duration ($p = 1.58 \times 10^{-3}$, $\delta = 0.314$, small) and the instrumentation-job envelope ($p = 4.29 \times 10^{-4}$, $\delta = 0.350$, small-to-medium). Community is therefore best interpreted not as a uniformly strong style, but as a *fast but variable* one: it often minimizes typical timing, yet it exposes developers to substantially less repeatable completion behavior. This main result is also supported by the study's two-tier workflow-shape robustness check, which applies the same comparison logic and statistical tests within more comparable workflow settings. In this follow-up, both Tier 1 and Tier 2 preserve the same fast-but-variable interpretation, with the strongest support concentrated in the Community-versus-GMD contrast. Detailed Tier 1 and Tier 2 support is reported in Tables 12, 13, 14, 15, and 16.

*Obs. 2.3 (Third-Party carries the strongest absolute tail-risk profile).* Third-Party forms the clearest *absolute tail-risk* profile in the current Base regime. This reading comes not only from variability relative to its own median, but also from the fact that its typical timing is already large before upper-tail escalation is considered. As shown in Table 4, Third-Party records a median overall duration of **1,864.5s**, a median instrumentation-job envelope of **1,717.5s**, and a median invocation-execution window of **1,881.5s**. Its upper-tail exposure is also substantial in absolute wall-clock terms, including a P95 total run duration of **34,603,064.5s**. Relative to GMD, Third-Party is both slower and less predictable on these major completion-oriented measures, with statistically significant differences and meaningful effect sizes. For example, its median normalized deviation is **0.256** versus **0.061** for run duration ($p = 7.95 \times 10^{-48}$, $\delta = 0.634$, medium-to-large), **0.233** versus **0.060** for the instrumentation-job envelope ($p = 2.64 \times 10^{-44}$, $\delta = 0.609$, medium-to-large), and **0.168** versus **0.045** for the invocation-execution window ($p = 3.93 \times 10^{-15}$, $\delta = 0.579$, medium-to-large). Relative to Community, Third-Party is not always worse on *normalized* variability, but it remains operationally riskier because its completion times are already much larger in absolute terms. Accordingly, when long delays are costly, Third-Party is the clearest style to interpret as high-risk.

*Obs. 2.4 (Custom remains a mixed predictability case).* Custom contributes a more mixed and cautious predictability profile. In Layer 1, it appears more stable than Community on broad completion-oriented measures, including run duration and the instrumentation-job envelope. Its median normalized deviation is **0.231** for run duration versus **0.506** for Community ($p = 1.58 \times 10^{-3}$, $\delta = -0.314$, small), and **0.283** for the instrumentation-job envelope versus **0.511** for Community ($p = 4.29 \times 10^{-4}$, $\delta = -0.350$, small-to-medium). However, this relative stability does not carry cleanly into the more precise Layer 2 view. On the directly observable invocation-centered measures, Custom shows a weaker and less favorable predictability signal: its median normalized deviation is **0.570** for pre-invocation and **0.680** for the invocation-execution window, both substantially larger than GMD's and also larger than Third-Party's. Accordingly, Custom is best interpreted as a mixed profile: moderately stable in the broad Layer 1 view, but less favorable once predictability is examined within the more precise execution-centered timing layer.

**RQ2 Summary (Predictability profile).** The four styles exhibit distinct predictability profiles under the controlled usable-verdict Base regime. GMD provides the strongest predictability-first profile and is the most stable style on the main completion-oriented measures. Community often delivers faster typical completion, but with much heavier variability. Third-Party carries the strongest absolute tail-risk profile because its already-large typical timing is coupled with substantial upper-tail exposure. Custom remains a mixed case: more stable than Community in the broad Layer 1 view, but less favorable in the more precise Layer 2 view. Taken together, these results show that predictability is not reducible to speed, and that different styles expose developers to different kinds of operational timing risk.

### *4.3. RQ3: What are the dominant sources of overhead by style?*

RQ3 examines how the four emulator-testing styles differ in *overhead placement* under the controlled Base regime. Rather than simply measuring how long runs take, it asks where observable delay accumulates and what visible work is most strongly associated with that delay in a form that remains interpretable for operational diagnosis. It treats **run duration** as the end-to-end baseline, uses Layer 1 to identify the broad placement of instrumentation-related time, and then uses Layer 2 to locate the dominant delay region more precisely through the three Layer 2 components: **Pre-Invocation**, **Invocation Execution Window**, and **Post-Invocation**. The resulting placement pattern is interpreted primarily through **step activity group**, while **execution role** is used as a supporting distinction where needed to clarify whether the visible work is mainly execution-related or non-execution overhead. Because the execution window is often dominated by *Test* activity, RQ3 does not treat that label by itself as the full overhead diagnosis. Instead, *Test* serves as an anchor for the core execution path, while the interpretive value of the step evidence comes mainly from how the non-execution burden is placed around that path and from the strongest visible non-test cues associated with it. Because RQ3 depends on directly observable Layer 2 timing, its share-based placement analysis is conducted on the **Layer 2-included Base subset** ($n$ = 5,057), that is, instrumentation-executed first-attempt usable-verdict records for which the study's Layer 2 timing components are directly available. Table 5 summarizes the resulting overhead-evidence matrix by style and provides the evidentiary basis for the observation-level diagnosis that follows.

Table 5: Overhead-evidence matrix by style under the Layer 2-included Base subset for RQ3 ($n$ = 5,057). Layer 2 share values are computed per run as each direct Layer 2 component divided by total run duration; because the three direct Layer 2 components sum to run duration for Layer 2-included records, these values are the within-run shares of run duration attributable to Pre-Invocation, Invocation Execution Window, and Post-Invocation. The table reports the median of those within-run shares by style.

| | **Layer 2 share evidence** | | | **Visible step evidence** | |
|---|---|---|---|---|---|
| **Style** | **Pre** | **Exec.** | **Post** | **Anchor activity in dominant region** | **Main non-test interpretive cue** |
| Community | 16.5% | 38.5% | 8.0% | Test | Artifact/Report; additional cleanup/setup |
| GMD | 6.8% | 92.7% | 0.4% | Test | Artifact/Report |
| Third-Party | 23.3% | 70.1% | 0.3% | Test | Pre-invoc. burden largely captured as coarse Other; very small post mainly Artifact/Report |
| Custom | 7.4% | 35.9% | 56.4% | Test | Post-invoc. burden dominated by coarse Other; limited explicit Artifact/Report |

*Note:* Visible step evidence summarizes the dominant visible work in the main delay region together with the strongest secondary non-test cue that helps interpret the placement pattern. Across styles, visible work in the invocation-execution window is predominantly execution-related, whereas visible work in pre-invocation and post-invocation segments is predominantly non-execution overhead. In some styles, especially Third-Party and Custom, that non-test burden is only coarsely observable in the current telemetry and is therefore interpreted cautiously.

*Obs. 3.1 (GMD provides the clearest execution-centric overhead profile).* GMD forms the clearest *execution-centric* overhead profile in the Layer 2-included Base subset. Its dominant burden sits inside the invocation-execution window rather than before invocation or after execution has nominally ended. As shown in Table 5, GMD's median invocation-execution share reaches **92.7%**, compared with **38.5%** for Community, **35.9%** for Custom, and **70.1%** for Third-Party. The contrast is especially strong relative to Community ($p = 2.15 \times 10^{-45}$, $\delta = 0.715$, large) and Third-Party ($p = 2.91 \times 10^{-29}$, $\delta = 0.828$, large). At the same time, GMD has a near-zero post-invocation share of only **0.4%**, with very strong separation from Community (**8.0%**; $p = 1.62 \times 10^{-55}$, $\delta = -0.794$, large) and clear separation from Custom (**56.4%**; $p = 1.79 \times 10^{-10}$, $\delta = -0.746$, large). The step-level evidence reinforces this interpretation. The execution window is anchored by *Test* work and is overwhelmingly *execution-related*, while the small residual post segment appears mainly as artifact/report-oriented follow-up rather than as a second meaningful bottleneck region. In profile terms, GMD is best read as the style whose observable burden is most cleanly concentrated in the core execution path, with little residual non-execution delay outside it. This main result is also strongly supported by the study's two-tier workflow-shape robustness check, which applies the same comparison logic and statistical tests within more comparable workflow settings. In this follow-up, both Tier 1 and Tier 2 preserve GMD as the clearest execution-centric style, with strong separation from Community and Third-Party. Detailed Tier 1 and Tier 2 support is reported in Tables 12, 13, 14, 15, and 16.

*Obs. 3.2 (Third-Party combines heavy entry with a still-dominant execution path).* Third-Party is best understood as a *heavy-entry plus heavy-execution* style rather than as a completion-tail- dominated one. As shown in Table 5, its median pre-invocation share reaches **23.3%**, far above GMD's **6.8%** ($p = 7.16 \times 10^{-27}$, $\delta = 0.791$, large), while its median invocation-execution share remains high at **70.1%**, clearly above Community's **38.5%** ($p = 9.77 \times 10^{-8}$, $\delta = 0.292$, small). By contrast, its post-invocation share remains negligible at only **0.3%**, well below Community's **8.0%** ($p = 8.35 \times 10^{-38}$, $\delta = -0.703$, large). The step-level evidence clarifies this pattern. The execution window is anchored by *Test* steps and is strongly *execution-related*, confirming that the style retains a substantial core execution burden. By contrast, the pre-invocation region is overwhelmingly *non-execution overhead*, although in the current telemetry it is captured mostly in the coarse *Other* activity bucket rather than in a more specific setup or provision label. Accordingly, the operational diagnosis is clear at the placement level even though the visible non-test labeling remains coarse: the main burden in Third-Party lies in getting into the execution path and then remaining there for a long time, not in a meaningful completion tail. This main result is also strongly supported by the study's two-tier workflow-shape robustness check, which applies the same comparison logic and statistical tests within more comparable workflow settings. In this follow-up, both Tier 1 and Tier 2 preserve Third-Party as a heavy-entry plus heavy-execution style rather than a completion-tail-dominated one. Detailed Tier 1 and Tier 2 support is reported in Tables 12, 13, 14, 15, and 16.

*Obs. 3.3 (Community shows a more distributed overhead profile).* Community does not concentrate observable delay as strongly as GMD or Third-Party in a single dominant bottleneck region. Instead, it shows a more *distributed overhead* profile. As shown in Table 5, its median invocation-execution share is **38.5%**, far below GMD's **92.7%** ($p = 2.15\times10^{-45}$, $\delta = -0.715$, large) and also below Third-Party's **70.1%** ($p = 9.77\times10^{-8}$, $\delta = -0.292$, small). At the same time, Community retains a materially larger post-invocation share than Third-Party (**8.0%** vs. **0.3%**, $p = 8.35\times10^{-38}$, $\delta = 0.703$, large), while showing a smaller entry burden than Third-Party (**16.5%** vs. **23.3%**, $p = 4.09\times10^{-8}$, $\delta = -0.300$, small). The step-level evidence explains this more distributed reading. Inside the execution window, the dominant visible work is *Test* and mainly *execution-related*, but the residual burden is not limited to that core path. The post-invocation segment is substantively populated by *Artifact/Report* work, with additional cleanup/setup presence, and is primarily *non-execution overhead*. In profile terms, Community is neither as purely execution-centric as GMD nor as entry-heavy as Third-Party. Instead, its delay is spread across meaningful execution work and a visible residual reporting/finalization phase, making it the clearest distributed case among the four styles.

*Obs. 3.4 (Custom remains the clearest tail-heavy mixed case).* Custom forms the clearest *tail-heavy mixed* profile in RQ3. In the pooled Layer 2 timing view, it shows the largest residual post-invocation interval among the four styles. As shown in Table 5, its median post-invocation share reaches **56.4%**, far above GMD's **0.4%** ($p = 1.79\times10^{-10}$, $\delta = 0.746$, large) and Third-Party's **0.3%** ($p = 8.53\times10^{-11}$, $\delta = 0.772$, large). At the same time, its median invocation-execution share remains well below GMD's (**35.9%** vs. **92.7%**, $p = 1.18\times10^{-14}$, $\delta = -0.903$, large) and also below Third-Party's (**35.9%** vs. **70.1%**, $p = 1.83\times10^{-4}$, $\delta = -0.445$, medium). The step-level breakdown shows that its execution-window steps are still anchored by *Test* and are mainly *execution-related*, but the visible post-invocation steps are sparse and captured largely in the coarse *Other* bucket rather than in a sharply defined artifact/report or cleanup pattern. This means that the main profile-level diagnosis remains clear—Custom's observable burden is dominated by a large residual post-execution segment—but the visible labels explain that tail only coarsely. Accordingly, Custom is best interpreted as a tail-heavy mixed case whose placement diagnosis is clear while its deeper activity-level explanation remains necessarily cautious.

**RQ3 Summary (Overhead-diagnosis profile).** The four styles exhibit distinct overhead-diagnosis profiles under the controlled usable-verdict Base regime. GMD provides the clearest execution-centric profile: its dominant burden sits inside the execution window, which is visibly test-centered and execution-related, with little meaningful residual delay outside it. Third-Party combines heavy entry with a still-dominant execution path rather than a meaningful completion tail; here the placement diagnosis is clear even though much of the visible pre-invocation burden is captured only coarsely. Community shows a more distributed profile in which both core execution work and a visible residual reporting/finalization phase matter. Custom remains the clearest tail-heavy mixed case, with a large post-execution burden that is visible in Layer 2 but only coarsely explained by the currently observable step labels. Taken together, these results provide the overhead-placement dimension of the overall operational profile by showing where delay accumulates, which parts of the run are most affected, and what visible work is most strongly associated with that burden at an interpretable level.

### *4.4. RQ4: How do emulator testing styles differ in CI deployment context and run-level verdict usability?*

RQ4 examines how the four emulator-testing styles differ in *deployment context* and *run-level outcome actionability* under the instrumentation-executed first-attempt regime. Unlike RQ1–RQ3, this analysis is not restricted to usable-verdict runs, because verdict usability is itself part of the outcome of interest. Using instrumentation-executed first-attempt run×style observations, it summarizes the distribution of CI trigger events, the distribution of run-level conclusions, the share of usable verdicts (`success`/`failure`) versus non-usable outcomes, and trigger-conditioned verdict patterns where relevant. Triggers are interpreted as deployment-context markers rather than as causal drivers of performance. The goal is therefore not only to observe whether a style finishes, but also whether it tends to produce an actionable terminal outcome, whether those usable outcomes are mostly successful or failure-dominant, and in what CI trigger contexts the style is most commonly deployed. Table 6 summarizes the compact profile by style, while the figures highlight the clearest outcome- and trigger-related contrasts.

Table 6: Compact RQ4 deployment-context and verdict-usability profile.

| Style | Usable | Success | Trigger |
|---|---|---|---|
| Community | 89.1% | 78.9% | push (88.0%) |
| GMD | 99.2% | 98.1% | schedule (90.2%) |
| Third-Party | 98.3% | 16.5% | schedule (87.3%) |
| Custom | 87.2% | 23.5% | pull_request (66.7%) |

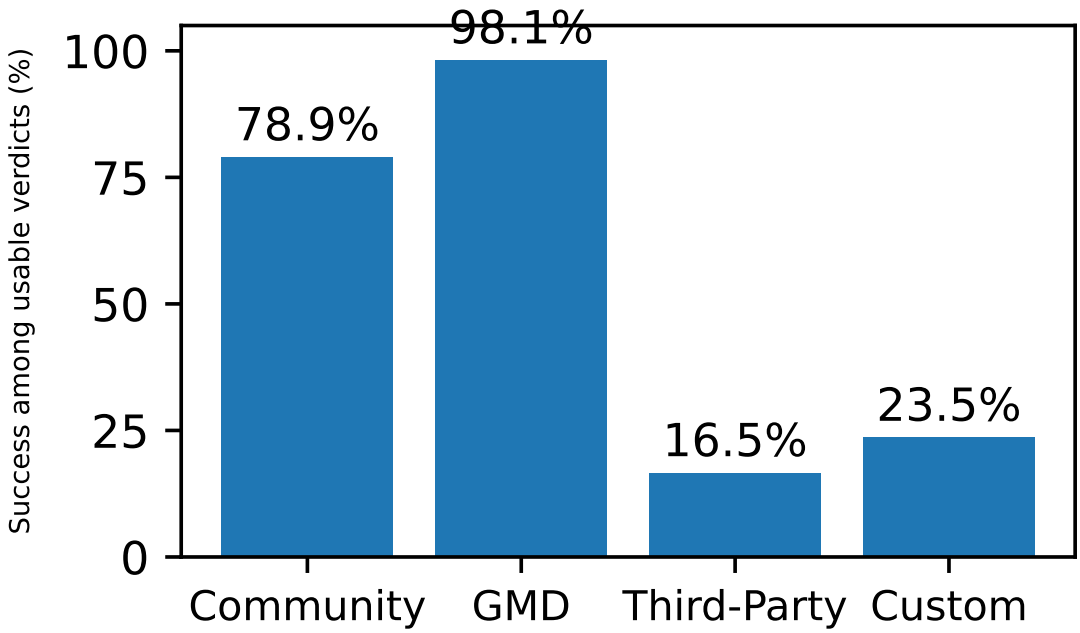


Figure 4: Success rate among usable verdicts by style under the RQ4 first-attempt regime. Usable verdicts are run-level terminal outcomes of `success` or `failure`.

*Obs. 4.1 (Usable-verdict rate differs across styles, but only modestly).* Under the instrumentation-executed first-attempt regime, the styles do not behave identically in their rate of usable run-level verdicts, but this separation is modest in practical terms. In the current snapshot, the overall association between style and usable-verdict status is statistically significant ($\chi^2$, $p = 9.33 \times 10^{-16}$), yet the effect size is small ($V = 0.092$). Descriptively, GMD shows the highest usable-verdict rate (**99.2%**), followed by Third-Party (**98.3%**), Community (**89.1%**), and Custom (**87.2%**). Thus, usable-verdict behavior should be treated as a real but weak differentiator across styles: some styles more often end in an actionable terminal outcome, but this dimension separates the styles much less strongly than the success-among-usable and deployment-context patterns reported later in RQ4.

*Obs. 4.2 (Among usable verdicts, the styles differ sharply in success orientation).* Conditioning on runs that reach a usable verdict reveals one of the clearest practice-facing separations in the study. As shown in Figure 4 and Table 6, GMD yields **98.1%** success among usable verdicts, Community yields **78.9%**, Custom yields **23.5%**, and Third-Party yields only **16.5%**. In the current snapshot, the overall association between style and success-versus-failure among usable verdicts is statistically significant ($\chi^2$, $p = 2.03 \times 10^{-247}$) with a strong effect size ($V = 0.384$). In profile terms, this means that the styles differ not only in whether they reach an actionable outcome, but also in the character of that outcome. GMD is overwhelmingly success-oriented once a usable verdict is produced. Third-Party and Custom are strongly failure-dominant, while Community occupies an intermediate but healthier position. This makes run-level outcome behavior an important complement to the timing profile: the styles differ not only in speed and predictability, but also in the success rate of the actionable outcomes they tend to produce. The appendix-level two-tier workflow-shape follow-up provides qualified support overall: the direction of the success-rate separation is

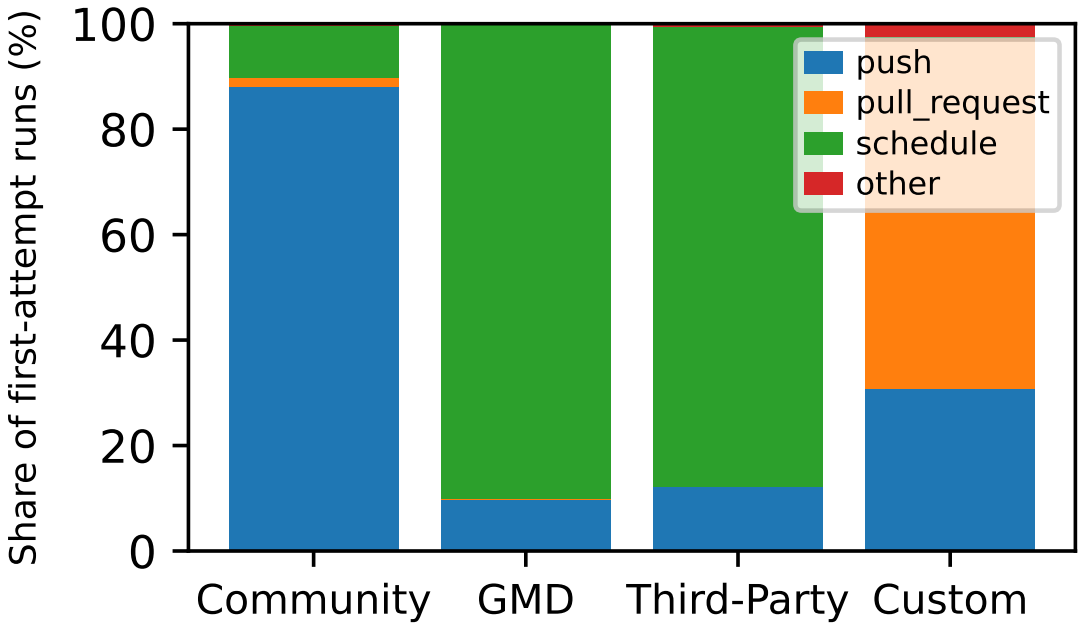


Figure 5: Trigger-event distribution by style under the RQ4 first-attempt regime. Each bar shows the within-style distribution of instrumentation-executed first-attempt runs across the main CI trigger contexts.

preserved across the split-anchor checks, with stronger support in the main paired family and weaker support in the secondary one (Tables 11, 12, 13, 14, 15, and 16).

*Obs. 4.3 (The styles are embedded in markedly different deployment contexts).* Deployment context is one of the clearest non-timing separations among the four styles. As shown in Figure 5 and Table 6, Community is used predominantly under `push` events, accounting for **88.0%** of its first-attempt observations. By contrast, GMD and Third-Party are both overwhelmingly associated with `schedule` events, at **90.2%** and **87.3%**, respectively. Custom differs from all other styles in being most strongly associated with `pull_request`, which accounts for **66.7%** of its observations. In the current snapshot, the overall association between style and trigger event remains statistically significant ($\chi^2$, $p \approx 0$ at machine precision) with a strong effect size ($V = 0.397$). This means that the styles should not be interpreted as interchangeable technical choices used in the same setting. Instead, they are embedded in meaningfully different CI deployment contexts in practice. The appendix-level two-tier workflow-shape follow-up provides strong robustness support for this interpretation: both eligible anchors and their paired broadened families preserve the same deployment-context separation, with Community aligned to `push`, GMD and Third-Party aligned to `schedule`, and Custom aligned to `pull_request` (Tables 12, 13, 14, 15, and 16).

*Obs. 4.4 (For some styles, especially Third-Party, outcome behavior is meaningfully trigger-conditioned).* For at least some styles, outcome behavior should not be treated as purely style-intrinsic. It is also meaningfully conditioned by the CI trigger context in which that style is deployed. The clearest case is Third-Party: among its usable verdicts, the association between trigger event and success-versus-failure is statistically significant ($\chi^2$, $p = 2.61 \times 10^{-83}$) with a very large effect size ($V = 0.850$). Descriptively, Third-Party usable verdicts under `push` are almost always successful (**98.5%**; 64/65), whereas under `schedule` they are overwhelmingly failure-oriented (**4.4%** success; 20/458). Custom shows the same directional pattern more cautiously and on much smaller support: the trigger-by-outcome association is statistically significant ($\chi^2$, $p = 6.93 \times 10^{-4}$, $V = 0.654$), with usable verdicts under `push` more often successful (**63.6%**; 7/11) than under `pull_request` (**4.5%**; 1/22). GMD also shows trigger-conditioned variation ($\chi^2$, $p = 1.35 \times 10^{-3}$, $V = 0.198$), but within a much healthier success-oriented range: usable verdicts under `push` are **87.5%** successful (21/24), compared with **99.2%** under `schedule` (237/239). Accordingly, for some styles—especially

Third-Party, and more cautiously Custom—outcome behavior is shaped not only by the style itself, but also by the deployment context in which it is used.

**RQ4 Summary (Deployment-context and actionability profile).** The four styles show distinct deployment-context and run-level outcome profiles under the instrumentation-executed first-attempt regime. Usable-verdict rate separates the styles only modestly, but success among usable verdicts and trigger context separate them much more clearly. GMD forms the strongest success-oriented profile. Community remains a healthier intermediate case. Third-Party and Custom are more failure-dominant among usable outcomes. Deployment context is also strongly differentiated: Community is mainly associated with `push`, GMD and Third-Party with `schedule`, and Custom with `pull_request`. In addition, for some styles—especially Third-Party, and more cautiously Custom—outcome behavior is meaningfully trigger-conditioned. Taken together, these results provide the deployment- and actionability-facing dimension of the overall operational profile.

## 5. Operational Performance Profile Summary

This section presents the paper's main synthesis output: an **operational information profile** for Android instrumentation-test execution styles in GitHub Actions. Rather than treating the empirical findings as isolated performance results, we consolidate them into a compact representation that supports comparison, interpretation, and decision making. The output has two parts: a style-level profile and a profile-derived decision-support guide. The profile is meant to support **objective-conditioned** interpretation rather than a single global ranking, which is especially important for styles such as **Third-Party**, whose pooled profile is relatively weak on the main timing and predictability dimensions but whose observed use appears concentrated in a narrower deployment context. The refreshable artifact companion preserves both outputs so that they can be regenerated on later telemetry snapshots under the same analytical logic [5].

### *5.1. Operational information profile*

Taken together, RQ1–RQ4 show that the four execution styles are better understood as distinct operational profiles than as members of a single global ranking. Each profile combines four information dimensions: timeliness, predictability, overhead placement, and deployment/outcome context.

**Community** forms the fastest typical completion profile, but not the most stable one. Its main strength lies in rapid completion through the execution-centered middle of the run, while its main limitation is higher variability and a more distributed overhead pattern. It is therefore best read as a **fast but variable baseline**: useful when typical completion time is the primary objective, but less attractive when stability or tail control matters more.

**GMD** forms the clearest predictability-first profile. It reaches the instrumentation path earliest, exhibits the tightest completion-oriented dispersion, and concentrates most observable time in the execution path itself. It is therefore best read as an **execution-centric stability profile**: not always the fastest to finish overall, but the most consistent and the most operationally stable.

**Third-Party** forms the clearest slow-path and absolute tail-risk profile. Its penalties accumulate both before invocation and during the main execution window, while its completion tail is not the dominant issue. It is therefore best understood not as a general-purpose default, but as a **context-sensitive provider-backed profile** whose observed use appears concentrated in a narrower deployment context and whose behavior is marked by entry delay, long execution windows, and trigger-conditioned outcome patterns.

**Custom** remains a sparse mixed case. It can enter the execution path relatively early, but its main distinguishing feature is a substantially heavier completion tail. Because its visible post-execution step labels are comparatively weak, its deep diagnosis remains cautious. It is therefore best read as a **tail-heavy repository-specific profile** rather than a general-purpose default.

Table 7 consolidates this profile state into a compact information product by style.

The main implication of this profile is that style choice should be **objective-driven rather than rank-driven**. The value of the profile lies in making trade-offs visible: fast completion, stability, bottleneck location, and actionability of outcomes do not align in the same way for all styles.

Table 7: Compact operational profile by style under the Step 3-validated controlled regime. The table summarizes the study's main information product rather than a single ranking.

| Style | Timeliness profile | Predictability | Dominant overhead reading | Outcome/deployment context |
|---|---|---|---|---|
| Community | Fastest typical completion; strong fast-core profile, but not uniformly fast across all phases. | Fast but variable; highest normalized variability on the main completion-oriented measures. | Distributed overhead, with both execution work and residual reporting/finalization mattering. | Usually yields usable verdicts with a comparatively healthy success rate; used mainly on `push`. |
| GMD | Clearest fast-entry profile; slower than Community in end-to-end completion, but stable and execution-focused. | Most predictable style on the main completion-oriented measures. | Clearest execution-centric profile, with near-zero residual tail. | Most often yields usable and successful outcomes; used mainly on `schedule`. |
| Third-Party | Slowest sustained-execution profile, with heavy entry burden and the longest execution window. | Strongest absolute tail-risk profile. | Heavy entry plus heavy execution, rather than a completion-tail-dominated pattern. | Usually yields a usable verdict, but among usable outcomes is strongly failure-dominant overall; used mainly on `schedule` and strongly trigger-conditioned. |
| Custom | Mixed-speed case: relatively early entry and better mid-path behavior than Third-Party, but much heavier completion tail. | Mixed and cautious profile: moderate Layer 1 stability, weaker Layer 2 stability, and sparse support. | Tail-heavy mixed case with a large post-invocation burden that is only coarsely explained by visible step labels. | Sparse, low-success profile in the observed sample; appears mainly on `pull_request`. |

### *5.2. Profile-derived decision-support guide*

The second synthesis output translates the profile into a small, interpretable **decision-support guide**. In the paper, this guide is a snapshot-grounded decision aid derived directly from the operational profile. In the artifact, the same guide is preserved in refreshable form so that it can be regenerated on later telemetry snapshots under the same analytical logic [5].

The guide is intentionally **profile-derived** rather than learned. The recommended style under each primary objective is taken from the empirical profile synthesis itself: RQ2 drives predictability-oriented recommendations, RQ1 drives fastest-typical-completion and fast-entry recommendations, RQ3 supplies the dominant bottleneck reading, and RQ4 informs outcome- and context-sensitive recommendations. This keeps the decision logic inspectable and close to the telemetry evidence, rather than collapsing the paper's results into a single opaque ranker.

Table 8 presents the resulting snapshot-level guide. Its purpose is not to impose a universal winner, but to help readers match a primary operational objective to the style whose profile best fits that objective in the current snapshot. For some objectives, the best-fit interpretation is broadly performance-oriented (for example, GMD for predictability-first use or Community for fastest typical completion). For others, the best-fit interpretation is explicitly **context-sensitive**, as with Third-Party and Custom, whose practical roles are narrower and should not be read as general-purpose defaults.

The paired optimization targets should be interpreted as **evidence-grounded starting points**, not experimentally validated remedies. They identify the most plausible first improvement direction suggested by the observed profile and bottleneck evidence, and therefore provide a practical bridge to future intervention-oriented work. For GMD, the natural target is the execution path itself [17]. For Community, the main target is to reduce setup variability and execution-path cost [40]. For Third-Party, the most relevant target is to reduce entry/provisioning delay and provider-side execution cost [18, 19]. For Custom, the most useful target is to reduce bespoke completion-tail work and standardize custom orchestration where feasible.

We therefore prefer an interpretable guide over a learned style selector. In high-cost decision settings, interpretable support is often more useful than opaque recommendation logic [22, 23, 41, 42]. Given the imbalance of the present corpus and the sparser support for some styles, a learned classifier would risk collapsing toward majority-style recommendations and would provide less transparent justification than the profile itself.

Table 8: Profile-derived decision-support guide for the current snapshot. The guide is objective-conditioned rather than rank-based: it matches each primary operational objective to the style whose observed profile best fits that objective in the current dataset, together with the most plausible first optimization target suggested by the evidence.

| Primary objective | Best-fit style | Profile-derived interpretation | Primary optimization suggestion |
|---|---|---|---|
| Most predictable feedback | GMD | Best fit when repeatable timing and low tail exposure matter most. GMD provides the clearest predictability-first profile and the tightest completion-oriented behavior in the current snapshot. | Reduce cost inside the main execution path; prioritize execution-window efficiency and repeatable execution configuration [17]. |
| Fastest typical completion | Community | Best fit when minimizing typical end-to-end feedback time matters more than minimizing variability. Community provides the fastest completion-oriented profile, but with materially higher variability than GMD. | Reduce setup variability and execution-path cost; prioritize caching, dependency reuse, and stabilization of the main execution path [40]. |
| Fastest entry into the instrumentation path | GMD | Best fit when reaching the instrumentation path as early as possible is the primary objective. GMD provides the clearest fast-entry profile, although it does not finish fastest overall. | Shorten the execution path after entry; prioritize in-window efficiency rather than additional entry-stage optimization [17]. |
| Strongest usable and successful outcome profile | GMD | Best fit when stable operational outcomes matter alongside timing. GMD combines the highest usable-verdict rate with the strongest success-oriented profile among usable outcomes in the current snapshot. | Focus on execution-path simplification and reduction of avoidable execution instability [17]. |
| Context-sensitive external-capacity / provider-backed use | Third-Party | Best interpreted not as a general-purpose default, but as a narrower provider-backed profile whose observed use is concentrated in a distinct deployment context. Its main burdens lie in entry delay, long execution windows, and strongly trigger-conditioned outcome behavior. | Reduce entry/provisioning delay and provider-side execution cost [18, 19]. |
| Repository-specific workflow constraints | Custom | Best interpreted as a narrow exception-path profile when more reusable Community-, GMD-, or provider-backed workflows are not feasible. Its profile is mixed, with relatively competitive entry but a much heavier completion tail and sparse support in the current snapshot. | Reduce project-specific completion-tail work and standardize custom orchestration where feasible, as suggested by the style's heavy completion tail and tail-heavy overhead profile in RQ1 and RQ3. |

### *5.3. Refreshable profile maintenance through the artifact companion*

Beyond the paper snapshot, the refreshable artifact companion preserves the method as an operational-information maintenance layer rather than only as a reproducibility aid. Its role is to reapply the same staged transformation from workflow telemetry to operational profile outputs on later snapshots under the same analytical logic [5].

Operationally, the artifact translates the paper's reported observations into a maintained set of short **operational profile questions**. These questions are derived from the paper's empirical findings and synthesis outputs and are indexed to the paper's observation structure. In the current snapshot, the paper provides the baseline answers to those questions. In later quarterly refreshes, the same questions are re-evaluated on updated telemetry so that the style profiles, decision-support outputs, and robustness companions can be refreshed without redefining the study method.

This refreshability matters because public GitHub Actions telemetry is historically bounded and retention-constrained, so any single paper snapshot provides only a limited temporal window on observed behavior. The artifact therefore helps extend the practical life of the method by preserving not just the data-processing pipeline, but the operational question structure through which telemetry is converted into interpretable and updateable operational information. Appendix 9.3 summarizes this maintained question layer in compact form.

## 6. Implications

The implications of this study follow from treating CI telemetry as an **operational information resource** rather than only as an execution log. In this view, the main contribution is not merely a set of style comparisons, but a method for transforming workflow traces into interpretable information products that support comparison, diagnosis, and decision making. The contribution is therefore relevant not only to software engineering practice, but also to research on information processing, interpretable analytics, operational information design, and telemetry-based decision support.

### *6.1. Implications for operational information design*

**Telemetry requires information design, not only data collection.** The study shows that raw workflow traces become more useful when they are transformed into structured operational profiles. For telemetry-rich workflow

environments, this means that a useful information product should preserve multiple decision-relevant dimensions—including feedback timeliness, predictability, overhead placement, and outcome actionability—rather than collapsing workflow behavior into a single end-to-end runtime measure.

**Interpretability is part of the information value.** The contribution of the proposed profile lies not only in summarizing behavior, but in making trade-offs legible to human decision makers. This aligns with broader work on interpretable analytics and explainable decision support, which argues that data-driven systems are more useful when their outputs remain inspectable and actionable [22, 23]. In telemetry-rich workflow settings, explanation should therefore be treated as a first-class design property of the information product rather than as a secondary presentation layer.

**Operational context should remain visible in the representation.** The findings show that observed style behavior is shaped not only by execution mechanism, but also by attempt regime, workflow shape, and trigger context. Operational information products should therefore preserve relevant contextual conditions instead of presenting performance values as if they were universally comparable. This is especially important in observational telemetry settings, where the usability of a comparison depends on how clearly the surrounding conditions are made visible.

### *6.2. Implications for practitioners and workflow owners*

**Style selection should begin from the operational objective.** The four studied styles are not interchangeable and do not form a single universal ranking. Community is best read as the fastest typical completion profile, GMD as the strongest predictability-first and fast-entry profile, Third-Party as a narrower provider-backed profile with stronger slow-path and tail-risk exposure, and Custom as a repository-specific mixed case with a much heavier completion tail. In practice, this means that workflow owners should begin from their primary objective—for example, faster typical feedback, stronger predictability, bottleneck diagnosis, or outcome quality—and then interpret the style profile accordingly.

**Intervention should target the dominant delay region.** The study shows that styles differ not only in how long they take, but also in where observable delay accumulates. Community shows a more distributed burden, GMD is the clearest execution-centric case, Third-Party combines heavy entry with a still-dominant execution path, and Custom is the clearest tail-heavy mixed case. This makes targeted intervention more plausible than general attempts to "speed up CI" without diagnosing where the burden actually sits.

**Outcome context is part of operational usefulness.** The outcome analysis shows that styles differ in usable-verdict rate, success among usable outcomes, and deployment context. Accordingly, style choice should not be evaluated only in timing terms. It should also be assessed in the specific operational setting where it will be used, such as push-time feedback, pull-request gating, or scheduled regression monitoring. A style that appears acceptable on runtime alone may still be a poor operational choice if it behaves weakly in the trigger regime or outcome profile that matters most in practice.

### *6.3. Implications for research*

**Operational profiling can serve as a reusable information-processing design.** The paper contributes a method for converting workflow evidence and run/job/step telemetry into an interpretable operational representation. Although instantiated here on Android instrumentation testing, the same general design could inform other telemetry-rich digital work environments in which raw traces must be transformed into structured operational knowledge.

**Refreshable artifacts can preserve analytical information products over time.** Because CI telemetry is retention-bounded and workflow ecosystems evolve, a static paper is only part of the information contribution. The refreshable artifact companion illustrates how the same analytical logic can preserve and regenerate operational profiles and profile-derived decision-support outputs on later telemetry snapshots. More generally, this suggests that analytical artifacts in telemetry-based research can be designed not only for reproducibility, but also for continuing operational information maintenance.

**Performance analysis should be combined with actionability analysis.** The results reinforce that timing alone is an incomplete operational view. Usable-verdict rate, success among usable outcomes, and deployment context all shape whether a workflow configuration is actually useful in practice. Future studies in related settings should therefore combine efficiency-oriented measures with actionability-oriented ones so that the resulting outputs better reflect how telemetry is used in real decision environments.

## 7. Threats to Validity

We discuss threats to construct, internal, external, and conclusion validity in light of the current study design, following established empirical software engineering guidance [29, 43].

### *7.1. Construct Validity*

*Constructing operational measures from CI telemetry.* The study derives timing, overhead, and outcome constructs from GitHub Actions run/job/step telemetry rather than from raw execution logs. This enables scalable historical analysis, but it also means that the results depend on the semantic accuracy of API-exposed timestamps, boundaries, and step records. In particular, Layer 1 and Layer 2 rely on selected cut points and matched execution anchors that may not perfectly reflect the underlying execution reality in all cases. We mitigate this threat through staged validation, source-traceability checks, and manual auditing.

*Historical telemetry interpreted through latest workflow contents.* The pipeline combines historical telemetry with the latest available workflow contents and repository-local references. Accordingly, some older runs may be interpreted through a later workflow snapshot rather than the exact workflow state that produced them. We mitigate this threat by grounding the dataset primarily in observed telemetry, using manual auditing, and treating workflow shape as a downstream robustness layer.

*Instrumentation-executed detection under incomplete step telemetry.* Stage 3 is intended to retain only run×style records in which instrumentation testing actually executed. However, it starts from Stage 2 records with positive instrumentation-job evidence, and when step telemetry is unavailable it cannot always verify whether the instrumentation steps truly ran. Thus, a record may remain in the extracted instrumentation-executed set even if the actual instrumentation steps were skipped due to gating or conditional logic. We mitigate this threat by auditing execution evidence where observable and by restricting the more precise Layer 2-based analyses to the directly step-observable subset.

*Style and overhead labeling as analytical abstractions.* Execution style, activity group, execution role, and overhead phase are inferred analytically rather than self-declared by repositories. Some workflows combine multiple mechanisms, and some steps serve mixed purposes. We therefore interpret these labels as evidence-grounded operational abstractions rather than exact causal categories, and we mitigate mislabeling risk through conservative rules and manual audit support.

### *7.2. Internal Validity*

*Confounding from non-random style selection..* Projects do not adopt Community, Custom, GMD, or Third-Party styles at random. Style choice may be associated with project size, test-suite scale, device requirements, team expertise, or organizational constraints, which may also affect timing and outcome behavior. We mitigate this threat by using a controlled timing-comparison regime, separating RQ4 from the timing Base, and treating workflow shape as a downstream robustness layer rather than as a full control design.

*Infrastructure variability and reruns.* GitHub-hosted CI runs are subject to queueing, transient slowdowns, and shared-runner variability. Reruns may also reflect failure recovery rather than ordinary first-attempt behavior. We mitigate these threats by emphasizing medians, non-parametric comparisons, tail-aware summaries, and by treating reruns as a separate adjacent regime rather than pooling them with the main Base timing subset.

### *7.3. External Validity*

*Platform, sample, and provider specificity.* The study focuses on GitHub Actions, Android instrumentation testing, and an observed corpus of publicly visible repositories. The results may not generalize directly to other CI platforms, mobile ecosystems, repository populations, or provider-backed device-farm services. We therefore interpret the empirical profile as snapshot-specific, while expecting the broader telemetry-driven profiling design to transfer more readily than the exact style-level results.

*Fixed repository universe in the refreshable artifact.* The refreshable artifact re-applies the method to later telemetry snapshots, but it starts from the same fixed input frame of repositories detected earlier as potentially instrumentation-capable. It does not automatically rediscover new relevant repositories or remove repositories whose instrumentation capability later disappears. Thus, the artifact supports refresh within a stable study population rather than ecosystem-wide re-sampling.

### *7.4. Inference and Conclusion Validity*

*Heavy tails, multiple comparisons, and observational interpretation.* CI timing data are often heavy-tailed, and the study compares multiple styles across multiple outcomes and robustness views. We mitigate these risks by using robust summaries, non-parametric tests, effect sizes, and correction for pairwise comparisons. More broadly, the operational profile and profile-derived decision-support guide should be read as evidence-grounded observational synthesis outputs, not as experimentally validated causal recommendations.

## 8. Conclusion

This paper treated CI telemetry as an information problem as well as a software-execution problem. Using Android instrumentation testing in GitHub Actions as the empirical setting, it presented a telemetry-driven method for transforming workflow traces into interpretable operational information. The method combines workflow evidence with run, job, and step telemetry to construct a study-facing run×style dataset, represent elapsed time through two aligned timing layers, separate comparable execution regimes, and compare execution styles through four operational information dimensions: feedback timeliness, predictability, overhead placement, and deployment/outcome context.

The empirical results showed that the four studied styles form distinct operational profiles rather than a single universal ranking. Community provides the fastest typical completion but with higher variability. GMD provides the strongest predictability-first and fast-entry profile. Third-Party carries the strongest absolute tail-risk profile, shaped by heavy entry and long execution windows. Custom remains a sparse mixed case with relatively early entry but a much heavier completion tail. The styles also differ in usable- verdict rate, success among usable outcomes, and deployment context.

The main synthesis contribution is therefore twofold. First, the paper consolidates the findings into an interpretable operational information profile for Android instrumentation-test execution styles. Second, it derives a decision-support guide that keeps recommendation logic close to the underlying telemetry evidence. Beyond the paper snapshot, a refreshable artifact companion preserves the same analytical transformation so that these information products can be regenerated on later telemetry snapshots under the same method [5].

More broadly, the study shows that telemetry-rich workflow environments benefit from representations that are interpretable, context-sensitive, and refreshable. The contribution is therefore not only an empirical comparison of Android CI execution styles, but also a methodological example of how raw workflow traces can be transformed into actionable operational information.

## 9. Appendix

### *9.1. Workflow-shape robustness eligibility and coarsened-family construction*

This subsection documents the construction of the study's two-tier workflow-shape robustness layer. It defines the eligibility rule for Tier 1 exact-signature follow-up, the paired-family construction rule for Tier 2, and the resulting eligible-signature coverage summarized in Table 9.

Tier 1 exact-signature follow-up is restricted to signatures that satisfy the study's minimum-support and usable cross-style-overlap requirements. In the current study, an exact signature is eligible only if it contains at least 80 total records, at least 15 records for each usable included style, and usable support for at least two styles under the relevant study regime.

Tier 2 then defines a paired coarsened family around each eligible exact signature by holding the runner-OS bucket fixed and broadening the executed job-count and executed step-count buckets to adjacent ordered categories. This paired-family construction is fixed per eligible exact signature and is used as the study's broader within-shape robustness layer.

Table 9: Eligible exact signatures and paired Tier 2 coarsened families used in the workflow-shape robustness layer.

| Eligible signature | Executed shape | Usable styles | Tier 1 | Tier 2 |
|---|---|---|---|---|
| 88f32b360855c277 | ubuntu / 1 / <=20 | Community, GMD, Third-Party | 874 | 3,168 |
| 0bc0e933a2435166 | mixed_or_unknown / >6 / >80 | Community, Third-Party, Custom | 1,223 | 1,479 |

In the current snapshot, two exact signatures satisfy the Tier 1 eligibility rule. The first eligible signature, `88f32b360855c277`, corresponds to the executed shape `ubuntu / 1 / <=20`. Its Tier 2 paired coarsened family holds the runner-OS bucket fixed at `ubuntu`, broadens the executed job-count bucket from `1` to {`1`, `2_3`}, and broadens the executed step-count bucket from `<=20` to {`<=20`, `21_40`}. The second eligible signature, `0bc0e933a2435166`, corresponds to the executed shape `mixed_or_unknown / >6 / >80`. Its Tier 2 paired coarsened family holds the runner-OS bucket fixed at `mixed_or_unknown`, broadens the executed job-count bucket from `>6` to {`4_6`, `>6`}, and broadens the executed step-count bucket from `>80` to {`41_80`, `>80`}.

Table 9 summarizes the eligible exact signatures, their usable Tier 1 style support, their Tier 1 exact-support counts, and the corresponding Tier 2 coarsened-family support counts.

### *9.2. Detailed Validation Rules and Additional Results*

This appendix provides the supporting detail for the validation procedure summarized in Section 3.5. In the main paper, Step 1 reports a compact category-level summary of automated representation validation, while Step 3 reports the high-level outcome of the robustness analysis. Here we provide the corresponding appendix-level validation detail: the full Step 1 rule set, the explicit timing-equation checks, the interpretation of telemetry-dependent rules, the detailed rule-level results in Table 10, the quantitative support for Validation Step 3 in Table 11, and the observation-level robustness follow-up in Tables 14– 16.

#### *9.2.1. Step 1 purpose and scope*

The goal of Step 1 is to establish representation integrity before the RQ-specific analyses begin. In particular, the automated checks verify that: (i) the constructed run×style records are uniquely and correctly keyed, (ii) required identifiers and retained study-facing fields are present where expected, (iii) run boundaries and selected cutpoints obey the expected temporal ordering, (iv) the Layer 1 and Layer 2 timing decompositions are internally coherent, (v) the selected invocation and execution-end cutpoints can be traced back to unique step-level source records, and (vi) the retained support fields used for comparability, interpretation, and robustness are present and internally consistent.

As in the main paper, we report the automated checks for both the full dataset and for Base, the study's main controlled timing regime defined by instrumentation-executed first-attempt usable-verdict records. Under the revised Step 1 design, the reported rates are *scope-based in-scope accuracies* rather than applicable-record accuracies. That is, each rule is evaluated on the subset where that rule should be available under the study's own measurement design, and the reported rate is computed as:

$$\frac{\texttt{ok}}{\text{rows in scope}}.$$

This appendix provides the rule-level detail behind that summary.

*Detailed rule set.* The automated validation comprised 54 rules:

1. **V1 Key uniqueness**: verifies that each record is uniquely identified by the study key combination.
2. **V2 Required identifiers present**: checks the presence of required identifiers such as workflow identifier, workflow ID, and workflow path.
3. **V3 Run-bounds order**: checks that run start time is less than or equal to run end time.
4. **V4 Window-bounds order**: checks that execution-window start time is less than or equal to execution-window end time.
5. **V5 Cutpoint temporal order**: checks that run start ≤ invocation start ≤ execution end ≤ run end.
6. **V6 Window inside run**: checks that the execution window lies within the overall run boundaries.

7. **V7 Run-duration non-negativity**: checks that total run duration is non-negative.
8. **V8 Layer 2 non-negativity**: checks that the three Layer 2 components are non-negative.
9. **V9 Components bounded by run duration**: checks that each Layer 2 component does not exceed total run duration.
10. **V10 Layer 1 sum-to-run equation**: checks that the sum of the three Layer 1 components equals run duration.
11. **V11 Layer 2 sum-to-run equation**: checks that the sum of the three Layer 2 components equals run duration.
12. **V12 Pre-invocation recomputation**: checks that the stored pre-invocation duration matches recomputation from timestamps.
13. **V13 Invocation-window recomputation**: checks that the stored invocation execution window matches recomputation from timestamps.
14. **V14 Post-invocation recomputation**: checks that the stored post-invocation duration matches recomputation from timestamps.
15. **V15 Unique invocation cutpoint selection**: checks that exactly one invocation cutpoint is selected from the step-level source data.
16. **V16 Unique execution-end cutpoint selection**: checks that exactly one execution-end cutpoint is selected from the step-level source data.
17. **V17 Invocation-step source match**: checks that the stored invocation-step fields match the selected source step record.
18. **V18 Execution-end-step source match**: checks that the stored execution-end-step fields match the selected source step record.
19. **V19 Style-scope validity**: checks that each record belongs to one of the study styles included in scope.
20. **V20 Base-timing-regime flag presence**: checks that the study's `Base_timing_regime` flag is present.
21. **V21 Layer 2-available-in-base flag presence**: checks that the study's `Layer2_available_in_base` flag is present.
22. **V22 Step-telemetry flag presence**: checks that the study's `Step_telemetry` flag is present.
23. **V23 Style-level distinct-job-count presence**: checks that the retained style-level distinct-job-count field is present.
24. **V24 Style-level distinct-job-base-name-count presence**: checks that the retained style-level distinct-job-base-name-count field is present.
25. **V25 Style-level matrix-like-job-count presence**: checks that the retained style-level matrix-like-job-count field is present.
26. **V26 Style-level matrix-expansion-flag presence**: checks that the retained style-level matrix-expansion flag is present.
27. **V27 Style-level parallel-same-style-flag presence**: checks that the retained style-level parallel-same-style flag is present.
28. **V28 Style-level max-parallel-jobs presence**: checks that the retained style-level max-parallel-jobs field is present.
29. **V29 Style-level repeated-same-style-flag presence**: checks that the retained style-level repeated-same-style flag is present.
30. **V30 Invocation-candidate-total-count presence**: checks that the invocation-candidate total count is present.
31. **V31 Stage-1 anchor candidate count presence**: checks that the anchor-match candidate count is present.
32. **V32 Explicit-instrumentation candidate count presence**: checks that the explicit-instrumentation candidate count is present.
33. **V33 Custom-supported candidate count presence**: checks that the custom-supported candidate count is present.
34. **V34 Distinct invocation-candidate step-name count presence**: checks that the distinct invocation-candidate step-name count is present.
35. **V35 Distinct invocation-candidate job count presence**: checks that the distinct invocation-candidate job count is present.

36. **V36 Selected invocation-priority-source presence**: checks that the selected invocation-priority source is present.
37. **V37 Execution-window candidate count presence**: checks that the execution-window candidate count is present.
38. **V38 Execution-window distinct-job-count presence**: checks that the execution-window distinct-job count is present.
39. **V39 Cross-job execution-window-flag presence**: checks that the cross-job execution-window flag is present.
40. **V40 Study signature-hash presence**: checks that the retained `study_signature_hash` field is present only when `Step_telemetry` is available for that run×style record.
41. **V41 Study runner-OS-bucket presence**: checks that the retained `study_runner_os_bucket` field is present.
42. **V42 Study executed job-count-bucket presence**: checks that the retained `study_job_count_exec_bucket` field is present.
43. **V43 Study executed step-count-bucket presence**: checks that the retained `study_step_count_exec_bucket` field is present.
44. **V44 Job-base-name count bounded by distinct-job count**: checks that the job-base-name count does not exceed the distinct-job count.
45. **V45 Matrix-like-job count bounded by distinct-job count**: checks that the matrix-like-job count does not exceed the distinct-job count.
46. **V46 Max-parallel-jobs bounded by distinct-job count**: checks that the max-parallel-jobs value does not exceed the distinct-job count.
47. **V47 Distinct invocation-step-name count bounded by invocation-candidate total**: checks that the distinct invocation-step-name count does not exceed the total invocation-candidate count.
48. **V48 Distinct invocation-candidate job count bounded by invocation-candidate total**: checks that the distinct invocation-candidate job count does not exceed the total invocation-candidate count.
49. **V49 Invocation-candidate partition consistency**: checks that the candidate subcounts do not exceed the total invocation-candidate count.
50. **V50 Execution-window distinct-job count bounded by execution-window candidate count**: checks that the distinct-job count within the execution window does not exceed the total execution-window candidate count.
51. **V51 Matrix-expanded style implies repeated same-style structure**: checks logical consistency between matrix-expanded and repeated-same-style indicators.
52. **V52 Cross-job execution-window flag consistency**: checks that the cross-job execution-window flag agrees with the corresponding distinct-job count.
53. **V53 Parallel same-style flag consistency**: checks that the parallel-same-style flag agrees with the max-parallel-jobs field.
54. **V54 Selected invocation-priority-source validity and contextual coherence**: checks that the selected invocation-priority source takes a valid current pipeline value and is coherent with the selected invocation state and corresponding candidate-support fields.

*Validation of Layer 1 and Layer 2 timing equations.* The automated procedure explicitly validates the study's two-layer timing model. As described in the main paper, Layer 1 and Layer 2 both decompose the same run duration at different levels of resolution.

For **Layer 1**, validity is checked directly through Rule V10:

$$\begin{aligned}\text{Run Duration} = {} & \text{Time to Instrumentation Envelope} \\ & + \text{Instrumentation Job Envelope} \\ & + \text{Post-Instrumentation Tail.}\end{aligned} \tag{1}$$

For **Layer 2**, validity is checked through timestamp-based recomputation from the selected cutpoints and run boundaries:

$$\text{Pre-Invocation} = \text{Invocation Start} - \text{Run Start.} \tag{2}$$

$$\text{Invocation Execution Window} = \text{Execution End} - \text{Invocation Start}. \tag{3}$$

$$\text{Post-Invocation} = \text{Run End} - \text{Execution End}. \tag{4}$$

Rules V12–V14 compare these recomputed values to the stored Layer 2 durations, while Rules V5 and V6 verify the temporal admissibility of the underlying cutpoints. Together, these checks confirm that the Layer 2 decomposition is both temporally and arithmetically consistent with the recorded run boundaries and selected step-level anchors.

*Interpretation of scope and missing cases.* The revised Step 1 design evaluates each rule on the subset where that rule should be available under the study's own measurement logic.

For **Layer 1-related** and broad study-facing rules, the *Overall* view is measured on the full dataset ($n$ = 8,845) and the *Base* view is measured on the Base timing regime ($n$ = 7,744). For **Layer 2- and step-telemetry-dependent** rules, the *Overall* view is measured only on records with `Step_telemetry=True` ($n$ = 5,747), and the *Base* view is measured only on records with `Step_telemetry=True` and `Base_timing_regime=True` ($n$ = 5,057).

Under this design, a `missing` value inside the relevant scope is treated as part of the observed failure burden rather than being removed from the denominator. This is especially important for rules such as V10 and V16–V18, where the new scope-based rates make residual missingness visible even when there is no direct contradiction in the stored values.

The revised outputs show that the broad structure of Step 1 remains unchanged: structural integrity, temporal consistency, Layer 2 recomputation validity, study-facing field integrity, and auxiliary consistency are perfect in both Overall and Base. The only non-perfect category-level results arise in Layer 1 equation validity and source traceability. In particular, Layer 1 equation validity is 98.45% in Overall and 99.96% in Base, reflecting 137 in-scope missing cases overall and 3 in Base for Rule V10. Source traceability remains near-perfect, with scope-based averages of 99.75% in Overall and 99.79% in Base. These small residual deviations are confined to unique-cutpoint selection and source-match rules (V15–V18), while all other Step 1 checks remain perfect under the revised scope-based design.

*Detailed Step 1 results.* Table 10 reports the full rule-level results underlying the category summary shown in the main paper. Overall and Base values now report *scope-based in-scope accuracy*, that is, `ok / rows in scope` under the rule-specific scope definition described above.

### *9.3. Appendix 9.4. Artifact-maintained operational question inventory*

To make the refreshable role of the artifact more explicit, this appendix summarizes the maintained operational questions through which the artifact preserves the paper's main profile outputs over time. These questions are derived from the paper's observation structure rather than from the research questions alone. In the current paper snapshot, they are answered by the reported findings. In later quarterly refreshes, the artifact re-evaluates the same questions on updated telemetry snapshots under the same analytical logic [5].

Table 10: Detailed Step 1 automated validation results by rule. Overall and Base values report scope-based in-scope accuracy, computed as `ok / rows in scope`. For Layer 1 and broad study-facing rules, Overall is measured on the full dataset and Base on the Base timing regime. For Layer 2 and step-telemetry-dependent rules, Overall is measured on `Step_telemetry=True` and Base on `Step_telemetry=True` within Base.

| Rule | Description | Overall | Base |
|---|---|---|---|
| V1 | Key uniqueness | 100.00% | 100.00% |
| V2 | Required identifiers present | 100.00% | 100.00% |
| V3 | Run bounds ordered | 100.00% | 100.00% |
| V4 | Execution-window bounds ordered | 100.00% | 100.00% |
| V5 | Cutpoints in temporal order | 100.00% | 100.00% |
| V6 | Execution window lies within run bounds | 100.00% | 100.00% |
| V7 | Run duration non-negative | 100.00% | 100.00% |
| V8 | Layer 2 durations non-negative | 100.00% | 100.00% |
| V9 | Layer 2 components bounded by run duration | 100.00% | 100.00% |
| V10 | Layer 1 sum equals run duration | 98.45% | 99.96% |
| V11 | Layer 2 sum equals run duration | 100.00% | 100.00% |
| V12 | Pre-invocation recomputation | 100.00% | 100.00% |
| V13 | Invocation-window recomputation | 100.00% | 100.00% |
| V14 | Post-invocation recomputation | 100.00% | 100.00% |
| V15 | Exactly one invocation cutpoint selected | 99.86% | 99.88% |
| V16 | Exactly one execution-end cutpoint selected | 99.69% | 99.70% |
| V17 | Invocation-step fields match source step record | 99.77% | 99.88% |
| V18 | Execution-end-step fields match source step record | 99.69% | 99.70% |
| V19 | Style is within study scope | 100.00% | 100.00% |
| V20 | Base-timing-regime flag present | 100.00% | 100.00% |
| V21 | Layer 2-available-in-base flag present | 100.00% | 100.00% |
| V22 | Step-telemetry flag present | 100.00% | 100.00% |
| V23 | Style-level distinct job count present | 100.00% | 100.00% |
| V24 | Style-level distinct job base-name count present | 100.00% | 100.00% |
| V25 | Style-level matrix-like job count present | 100.00% | 100.00% |
| V26 | Style-level matrix-expansion flag present | 100.00% | 100.00% |
| V27 | Style-level parallel same-style flag present | 100.00% | 100.00% |
| V28 | Style-level max-parallel-jobs field present | 100.00% | 100.00% |
| V29 | Style-level repeated same-style flag present | 100.00% | 100.00% |
| V30 | Invocation-candidate count total present | 100.00% | 100.00% |
| V31 | Stage-1 anchor candidate count present | 100.00% | 100.00% |
| V32 | Explicit-instrumentation candidate count present | 100.00% | 100.00% |
| V33 | Custom-supported candidate count present | 100.00% | 100.00% |
| V34 | Distinct invocation-candidate step-name count present | 100.00% | 100.00% |
| V35 | Distinct invocation-candidate job count present | 100.00% | 100.00% |
| V36 | Selected invocation-priority source present | 100.00% | 100.00% |
| V37 | Execution-window candidate count present | 100.00% | 100.00% |
| V38 | Execution-window distinct-job count present | 100.00% | 100.00% |
| V39 | Cross-job execution-window flag present | 100.00% | 100.00% |
| V40 | Study signature hash present | 100.00% | 100.00% |
| V41 | Study runner-OS bucket present | 100.00% | 100.00% |
| V42 | Study executed job-count bucket present | 100.00% | 100.00% |
| V43 | Study executed step-count bucket present | 100.00% | 100.00% |
| V44 | Style job-base count ≤ distinct job count | 100.00% | 100.00% |
| V45 | Matrix-like count ≤ distinct job count | 100.00% | 100.00% |
| V46 | Max parallel jobs ≤ distinct job count | 100.00% | 100.00% |
| V47 | Distinct invocation-step-name count ≤ invocation-candidate total | 100.00% | 100.00% |
| V48 | Distinct invocation-candidate job count ≤ invocation-candidate total | 100.00% | 100.00% |
| V49 | Invocation-candidate partition consistency | 100.00% | 100.00% |
| V50 | Execution-window distinct-job count ≤ execution-window candidate count | 100.00% | 100.00% |
| V51 | Matrix-expanded implies repeated-style consistency | 100.00% | 100.00% |
| V52 | Cross-job execution-window flag consistency | 100.00% | 100.00% |
| V53 | Parallel same-style flag consistency | 100.00% | 100.00% |
| V54 | Selected invocation-priority-source validity and contextual coher- | 100.00% | 100.00% |

Table 11: Quantitative support for Validation Step 3 robustness checks.

| Check | Target | Quantitative support | Result |
|---|---|---|---|
| Controller regime | `all_run_per_style` vs. Base | Base ordering preserved exactly for **6/7** timing measures; **1/7** partially preserved | Largely preserved |
| Controller regime | first-attempt view vs. Base | Base ordering preserved exactly for **6/7** timing measures; **1/7** partially preserved | Largely preserved |
| Controller regime | `rerun_usable_verdict` vs. Base | Base ordering preserved for **0/7** measures; **3/7** partially preserved; **2/7** changed; **2/7** not testable | Distinct adjacent regime |
| Workflow shape | Exact-signature candidate pool | **20** candidate signatures identified; only **2** eligible for deeper follow-up; each provides only **3** usable styles, not a balanced four-style setting | Partially sufficient |
| Workflow shape | Selected exact-signature follow-up | Across the **2** eligible signatures (**14** within-signature timing checks), **0/14** reproduce the full Base ordering exactly; **8/14** are partially preserved; and **6/14** differ from the Base pattern | Informative, not standalone |
| Workflow shape | Selected paired-family follow-up | Across the **2** paired Tier 2 families (**14** within-family timing checks), **0/14** reproduce the full Base ordering exactly; **6/14** are partially preserved; and **8/14** differ from the Base pattern | Informative, not standalone |

Table 12: Tier 1 exact-signature robustness support for the selected key observations.

| Obs. | Tier 1 direction note | Tier 1 compact statistical support | Tier 1 result |
|---|---|---|---|
| 1.1 | Community remains lower than GMD and Third-Party on run duration and L2 execution window. | Support is clear relative to Third-Party; Community-vs-GMD remains weaker, especially on run duration. | Qualified |
| 1.2 | GMD remains earliest on L2 pre-invocation relative to Third-Party and remains not-fastest on completion; the Community-vs-GMD entry contrast is weaker. | Entry-side support is strong relative to Third-Party; completion-side support remains visible against Community. | Qualified |
| 1.3 | Third-Party remains slowest on run duration, L2 pre-invocation, and L2 execution window. | Key contrasts against Community and GMD remain strongly supported on the sustained-execution measures. | Strong |
| 2.1 | GMD remains the tightest style on the main completion-oriented predictability measures. | The predictability gap remains strong relative to Community and remains clearly visible relative to Third-Party. | Strong |
| 2.2 | Community remains fast in typical terms, but less stable than GMD on the main completion-oriented measures. | The speed–predictability tradeoff remains visible; support is strongest in the Community-vs-GMD contrast. | Supported |
| 3.1 | GMD remains the clearest execution-centric style, with high execution share and very small residual tail. | Execution-centric concentration remains strongly separated from Community and Third-Party. | Strong |
| 3.2 | Third-Party remains a heavy-entry plus heavy-execution style, not a completion-tail-dominated one. | Entry- and execution-side contrasts remain strongly supported against the tighter styles. | Strong |
| 4.2 | The strong success-rate separation remains visible, but no single exact signature supports a balanced four-style exact check. | Support is distributed across the two anchors rather than concentrated in one exact-signature comparison. | Partial |
| 4.3 | The trigger-context separation remains visible: Community aligns with push, GMD/Third-Party with schedule, and Custom with `pull_request`. | The deployment-context contrast remains strong across the two eligible exact-signature settings. | Strong |

*Note:* Tier 1 uses the selected eligible exact signature as a strict within-shape check. For Obs. 1.1–3.2, the Tier 1 anchor is `88f32b360855c277`. For RQ4 observations (Obs. 4.2 and Obs. 4.3), support is summarized as split-anchor because no single exact signature provides a balanced four-style setting.

Table 13: Tier 2 paired coarsened-family robustness support for the selected key observations.

| Obs. | Tier 2 direction note | Tier 2 compact statistical support | Tier 2 result |
|---|---|---|---|
| 1.1 | Community remains faster than Third-Party and remains lower than GMD on the main completion-oriented measures. | The completion-side contrasts become clearer than in Tier 1, especially on run duration and L2 execution window. | Supported |
| 1.2 | The fast-entry interpretation remains clear relative to Third-Party, but Community and GMD become close on the entry-side metric. | Completion-side contrast remains visible; entry-side support weakens in the closest broadened neighborhood. | Partial |
| 1.3 | The slow sustained-execution interpretation remains visible on run duration, L2 pre-invocation, and L2 execution window. | The key pairwise contrasts remain strongly supported in the broadened family. | Strong |
| 2.1 | GMD remains the tightest style on the main completion-oriented predictability measures. | Predictability support remains strong in the broadened family. | Strong |
| 2.2 | Community remains fast in typical terms but predictability-poor relative to GMD. | The speed–predictability tradeoff remains directionally visible and statistically supported in the broadened family. | Supported |
| 3.1 | GMD remains the clearest execution-centric style in the broader family. | Execution-share dominance and very small residual tail remain clearly supported. | Strong |
| 3.2 | Third-Party remains characterized by heavy entry and heavy execution. | The broadened family preserves the same overhead-placement interpretation. | Strong |
| 4.2 | Success-rate differences remain visible, but support remains distributed across the two broadened families rather than one unified Tier 2 setting. | Tier 2 broadening improves support only modestly because the observation remains structurally split. | Partial |
| 4.3 | The deployment-context pattern remains clearly visible in the paired broadened families. | The event-distribution contrast remains strongly supported across the two broadened settings. | Strong |

*Note:* Each Tier 2 family is defined from its Tier 1 anchor by holding the runner-OS bucket fixed and broadening executed job-count and executed step-count buckets to adjacent categories. For Obs. 1.1–3.2, Tier 2 uses the paired coarsened family derived from `88f32b360855c277`. For RQ4 observations (Obs. 4.2 and Obs. 4.3), support is summarized as split paired families because no single Tier 2 family provides a unified balanced four-style setting.

Table 14: Overall robustness reading for the selected key observations.

| Obs. | Observation-level robustness reading | Tier 1 | Tier 2 | Overall |
|---|---|---|---|---|
| 1.1 | Community remains the fastest completion-oriented style, with clearer support against Third-Party than against GMD. | Qualified | Supported | Qualified |
| 1.2 | GMD remains the clearest fast-entry style relative to Third-Party, but the Community–GMD entry contrast stays weak. | Qualified | Partial | Qualified |
| 1.3 | Third-Party remains the slowest sustained-execution style across the key timing measures. | Strong | Strong | Strong |
| 2.1 | GMD remains the tightest and most predictable style on the main completion-oriented measures. | Strong | Strong | Strong |
| 2.2 | Community remains fast in typical terms but substantially less predictable than GMD. | Supported | Supported | Supported |
| 3.1 | GMD remains the clearest execution-centric overhead profile, with a dominant execution share and minimal residual tail. | Strong | Strong | Strong |
| 3.2 | Third-Party remains a heavy-entry plus heavy-execution style rather than a completion-tail-dominated one. | Strong | Strong | Strong |
| 4.2 | The strong success-rate separation remains visible, but support is structurally split across anchors/families rather than unified in one balanced four-style setting. | Partial | Partial | Qualified |
| 4.3 | The deployment-context separation remains clearly visible across the robustness follow-up settings. | Strong | Strong | Strong |

Table 15: Tier 1 exact-signature statistical support for the selected key observations.

| Obs. | Metric | Contrast | $n_1$ / $n_2$ | Median$_1$ / Median$_2$ | $p$ / effect | Interpretation |
|---|---|---|---|---|---|---|
| 1.1 | Run duration | Community vs. GMD | 729 / 131 | 1122 / 1158 | $p = 8.13 \times 10^{-1}$, $\delta = -0.019$ | Direction is preserved, but the completion-side separation between Community and GMD is weak in the exact-signature setting. |
| 1.1 | L2 exec. window | Community vs. Third-Party | 729 / 81 | 892 / 1819 | $p = 5.22 \times 10^{-29}$, $\delta = -0.612$ | Strong completion-side support remains visible relative to Third-Party. |
| 1.2 | L2 pre-invocation | GMD vs. Third-Party | 131 / 81 | 79 / 561 | $p = 4.11 \times 10^{-34}$, $\delta = -0.949$ | Fast-entry support is extremely strong relative to Third-Party. |
| 1.2 | L2 pre-invocation | Community vs. GMD | 729 / 131 | 121 / 79 | $p = 2.64 \times 10^{-1}$, $\delta = 0.067$ | The Community-vs.-GMD entry contrast is weak in the exact-signature setting. |
| 1.3 | Run duration | Third-Party vs. Community | 81 / 729 | 2383 / 1122 | $p = 1.74 \times 10^{-21}$, $\delta = 0.678$ | Strong slow-path support for Third-Party is preserved in Tier 1. |
| 1.3 | L2 exec. window | Third-Party vs. GMD | 81 / 131 | 1819 / 1078 | $p = 1.17 \times 10^{-7}$, $\delta = 0.478$ | Sustained-execution disadvantage remains clearly visible against GMD. |
| 2.1 | Norm. abs. deviation (run duration) | GMD vs. Community | 131 / 729 | 0.061 / 0.513 | $p = 2.66 \times 10^{-100}$, $\delta = -0.771$ | Strong Tier 1 predictability support: GMD remains much tighter than Community. |
| 2.1 | Norm. abs. deviation (L2 exec. window) | GMD vs. Third-Party | 131 / 81 | 0.045 / 0.168 | $p = 3.93 \times 10^{-15}$, $\delta = -0.579$ | Predictability support remains clearly visible against Third-Party. |
| 2.2 | Norm. abs. deviation (run duration) | Community vs. GMD | 729 / 131 | 0.513 / 0.061 | $p = 2.66 \times 10^{-100}$, $\delta = 0.771$ | The fast-but-variable Community profile remains strongly preserved in Tier 1. |
| 3.1 | L2 execution share | GMD vs. Community | 131 / 729 | 0.927 / 0.385 | $p = 2.15 \times 10^{-45}$, $\delta = 0.715$ | Execution-centric separation remains very strong in the exact-signature setting. |
| 3.1 | L2 post-invocation share | Community vs. GMD | 729 / 131 | 0.080 / 0.004 | $p = 1.62 \times 10^{-55}$, $\delta = 0.794$ | Residual-tail contrast remains very strong against GMD. |
| 3.2 | L2 pre-invocation share | Third-Party vs. GMD | 81 / 131 | 0.233 / 0.068 | $p = 7.16 \times 10^{-27}$, $\delta = 0.791$ | Heavy-entry support for Third-Party remains very strong. |
| 3.2 | L2 execution share | Third-Party vs. Community | 81 / 729 | 0.701 / 0.385 | $p = 9.77 \times 10^{-8}$, $\delta = 0.292$ | Heavy-execution placement remains preserved in the exact-signature setting. |
| 4.2 | Success among usable verdicts | Style × success/failure | 874; 1223 | – | `88f32`: $p = 2.34 \times 10^{-58}$, $V = 0.364$; `0bc0`: $p = 3.11 \times 10^{-2}$, $V = 0.072$ | Tier 1 exact-signature support for the success-rate separation is strong in the main anchor and weaker in the secondary anchor. |
| 4.3 | Trigger event distribution | Style × event | 941; 1263 | – | `88f32`: $p = 4.02 \times 10^{-41}$, $V = 0.268$; `0bc0`: $p = 6.88 \times 10^{-154}$, $V = 0.494$ | The deployment-context separation remains strongly supported across both eligible exact-signature settings. |

*Note:* Tier 1 uses the eligible exact signature as a strict within-shape check. For Obs. 1.1–3.2, the Tier 1 anchor is `88f32b360855c277`. For RQ4 observations (Obs. 4.2 and Obs. 4.3), no single exact signature yields a balanced four-style setting, so the support is summarized as a split-anchor exact-signature view. MWU = Mann–Whitney; $\delta$ = Cliff's delta; $V$ = Cramer's V.

Table 16: Tier 2 paired coarsened-family statistical support for the selected key observations.

| Obs. | Metric | Contrast | $n_1$ / $n_2$ | Median$_1$ / Median$_2$ | $p$ / effect | Interpretation |
|---|---|---|---|---|---|---|
| 1.1 | Run duration | Community vs. GMD | 2950 / 131 | 750 / 1158 | $p = 2.73\times10^{-25}$, $\delta = -0.536$ | Broadened Tier 2 support strongly confirms faster Community on overall completion. |
| 1.1 | L2 exec. window | Community vs. Third-Party | 2713 / 87 | 286 / 1784 | $p = 1.05\times10^{-35}$, $\delta = -0.784$ | Strong completion-side support remains visible in the broadened family. |
| 1.2 | L2 pre-invocation | GMD vs. Third-Party | 131 / 87 | 79 / 561 | $p = 4.57\times10^{-35}$, $\delta = -0.989$ | Fast-entry support remains extremely strong relative to Third-Party. |
| 1.2 | L2 pre-invocation | Community vs. GMD | 2713 / 131 | 74 / 79 | $p = 3.36 \times 10^{-1}$, $\delta = -0.050$ | Entry-side support weakens in Tier 2 because Community and GMD become very close. |
| 1.3 | Run duration | Third-Party vs. Community | 87 / 2950 | 2333 / 750 | $p = 1.76\times10^{-39}$, $\delta = 0.826$ | Strong slow-path support for Third-Party is preserved in the broadened family. |
| 1.3 | L2 exec. window | Third-Party vs. GMD | 87 / 131 | 1784 / 1078 | $p = 2.30 \times 10^{-4}$, $\delta = 0.295$ | Sustained-execution disadvantage remains visible and statistically supported. |
| 2.1 | Norm. abs. deviation (run duration) | GMD vs. Community | 131 / 2950 | 0.042 / 0.441 | $p = 1.33\times10^{-50}$, $\delta = -0.771$ | Strong Tier 2 predictability support: GMD remains much tighter than Community. |
| 2.1 | Norm. abs. deviation (L2 exec. window) | GMD vs. Third-Party | 131 / 87 | 0.043 / 0.266 | $p = 5.62\times10^{-17}$, $\delta = -0.670$ | Predictability signal remains clearly visible against Third-Party. |
| 2.2 | Norm. abs. deviation (run duration) | Community vs. GMD | 2950 / 131 | 0.441 / 0.042 | $p = 1.33\times10^{-50}$, $\delta = 0.771$ | The fast-but-variable Community profile remains strongly preserved in Tier 2. |
| 3.1 | L2 execution share | GMD vs. Community | 131 / 2713 | 0.927 / 0.598 | $p = 1.30\times10^{-46}$, $\delta = 0.741$ | Execution-centric separation remains very strong in the broadened family. |
| 3.1 | L2 post-invocation share | Community vs. GMD | 2713 / 131 | 0.051 / 0.004 | $p = 3.75\times10^{-59}$, $\delta = 0.838$ | Residual-tail contrast remains very strong against GMD. |
| 3.2 | L2 pre-invocation share | Third-Party vs. GMD | 87 / 131 | 0.238 / 0.068 | $p = 1.21\times10^{-25}$, $\delta = 0.838$ | Heavy-entry support for Third-Party remains very strong. |
| 3.2 | L2 execution share | Third-Party vs. Community | 87 / 2713 | 0.760 / 0.598 | $p = 3.83 \times 10^{-5}$, $\delta = 0.259$ | Heavy-execution placement remains preserved in the broadened family. |
| 4.2 | Success among usable verdicts | Style × success/failure | 3168; 1479 | – | `88f32`: $p = 7.95 \times 10^{-130}$, $V = 0.433$; `0bc0`: $p = 2.17 \times 10^{-3}$, $V = 0.091$ | Tier 2 broadening supports the success-rate separation strongly in the main family and weakly in the secondary family. |
| 4.3 | Trigger event distribution | Style × event | 3314; 1660 | – | `88f32`: $p = 6.63 \times 10^{-110}$, $V = 0.281$; `0bc0`: $p = 1.54 \times 10^{-190}$, $V = 0.520$ | The deployment-context separation remains strongly supported across both broadened families. |

*Note:* Tier 2 uses the paired coarsened family derived from the observation's Tier 1 anchor. For Obs. 1.1–3.2, Tier 2 uses the paired coarsened family derived from `88f32b360855c277`. For RQ4 observations (Obs. 4.2 and Obs. 4.3), support is summarized as split paired families because no single Tier 2 family provides a unified balanced four-style setting. For RQ1–RQ3, the robustness check is conducted under the Base timing regime. For RQ4, the robustness check is conducted under the first-attempt outcome regime; Obs. 4.2 is additionally conditioned on usable verdicts. MWU = Mann–Whitney; $\delta$ = Cliff's delta; $V$ = Cramer's V.

Table 17: Artifact-maintained operational question inventory. The numbering links each maintained question to the paper's observation structure.

| ID | Maintained operational question |
|---|---|
| 1.1 | Which style provides the fastest completion-oriented timeliness profile in the current snapshot? |
| 1.2 | Which style reaches the instrumentation path earliest in the current snapshot? |
| 1.3 | Which style shows the slowest sustained-execution timeliness profile in the current snapshot? |
| 1.4 | Which style remains the clearest mixed timeliness case shaped by a heavy completion tail in the current snapshot? |
| 1.5 | Is Community's speed advantage concentrated mainly in the core execution path rather than uniformly across all timing segments in the current snapshot? |
| 2.1 | Which style provides the strongest predictability-first profile in the current snapshot? |
| 2.2 | Does Community still exhibit the clearest speed–predictability tradeoff in the current snapshot? |
| 2.3 | Which style carries the strongest absolute tail-risk profile in the current snapshot? |
| 2.4 | Does Custom remain a mixed predictability case in the current snapshot? |
| 3.1 | Which style provides the clearest execution-centric overhead profile in the current snapshot? |
| 3.2 | Does Third-Party still combine heavy entry with a still-dominant execution path in the current snapshot? |
| 3.3 | Does Community still show the clearest distributed overhead profile in the current snapshot? |
| 3.4 | Does Custom remain the clearest tail-heavy mixed overhead case in the current snapshot? |
| 4.1 | Which style has the highest usable-verdict rate in the current snapshot? |
| 4.2 | Which style has the strongest success-oriented usable outcome profile in the current snapshot? |
| 4.3 | Which style is most strongly associated with schedule-triggered deployment in the current snapshot? |
| 4.4 | Does Third-Party continue to show the clearest trigger-conditioned outcome behavior in the current snapshot? |
| 5.1 | Which style best fits a predictability-first operational objective in the current snapshot? |
| 5.2 | Which style best fits a fastest-typical-completion objective in the current snapshot? |
| 5.3 | Which style best fits a fast-entry objective in the current snapshot? |
| 5.4 | Which style best fits a context-sensitive provider-backed deployment objective in the current snapshot? |
| 5.5 | Which style best fits repository-specific workflow constraints in the current snapshot? |

## References


[1] GitHub Docs, Rest api endpoints for workflow runs, https://docs.github.com/rest/actions/workflow-runs, accessed 2026-02-12 (2026).

[2] GitHub Docs, Rest api endpoints for workflow jobs, https://docs.github.com/en/rest/actions/workflow-jobs, accessed 2026-02-12 (2026).

[3] T. Ghaleb, et al., An empirical study of the long duration of continuous integration builds, Empirical Software Engineering (2019). doi:10.1007/s10664-019-09695-9.

[4] H. Parsazadeh, T. A. Ghaleb, S. Hassan, Android instrumentation testing in continuous integration: Practices, patterns, and performance, in: Proceedings of the 19th IEEE International Conference on Software Testing, Verification and Validation (ICST), Daejeon, Republic of Korea, 2026, accepted paper.

[5] H. Parsazadeh, T. A. Ghaleb, S. Hassan, emulator-testing-operational-profiles: Refreshable artifact companion for operational information profiling of android instrumentation testing in github actions, https://github.com/behnamparsa/emulator-testing-operational-profiles, gitHub repository. Accessed 2026-03-30 (2026).

[6] K. Byström, K. Järvelin, Task complexity affects information seeking and use, Information Processing & Management 31 (2) (1995) 191–213. doi:10.1016/0306-4573(94)00041-Z.

[7] J. D. E. Johnson, D. O. Case, J. Andrews, S. L. Allard, N. E. Johnson, Fields and pathways: Contrasting or complementary views of information seeking, Information Processing & Management 42 (2) (2006) 569–582. doi:10.1016/j.ipm.2004.12.001.

[8] M. Hertzum, Expertise seeking: A review, Information Processing & Management 50 (5) (2014) 775–795. doi:10.1016/j.ipm.2014.04.003.

[9] E. Auster, C. W. Choo, How senior managers acquire and use information in environmental scanning, Information Processing & Management 30 (5) (1994) 607–618. doi:10.1016/0306-4573(94)90073-6.

[10] Android Developers, Build instrumented tests, https://developer.android.com/training/testing/instrumented-tests, accessed 2026-03-04 (2025).

[11] Android Developers, Set up your project for androidx test, https://developer.android.com/training/testing/instrumented-tests/androidx-test-libraries/test-setup, accessed 2026-03-04 (2025).

[12] Android Developers, Espresso, https://developer.android.com/training/testing/espresso, accessed 2026-03-04 (2025).

[13] A. Tahir, et al., Test flakiness' causes, detection, impact and responses, Journal of Systems and Software (2023).

[14] V. Pontillo, et al., Test code flakiness in mobile apps: The developer's perspective, Information and Software Technology (2024).

[15] A. Romano, W. Wang, A. Zaidman, M. Aniche, S. Panichella, An empirical analysis of ui-based flaky tests, in: Proceedings of the International Conference on Software Engineering (ICSE), 2021. doi:10.1109/ICSE43902.2021.00141.

[16] S. T. C. S. N. Meng, N. Nagappan, et al., An empirical study of flaky tests in android apps, in: Proceedings of the International Conference on Software Engineering (ICSE), 2019.

[17] Android Developers, Scale your tests with build-managed devices (gradle managed devices), https://developer.android.com/studio/test/managed-devices, accessed 2026-03-04 (2026).

[18] Firebase, Get started with instrumentation tests (firebase test lab), https://firebase.google.com/docs/test-lab/android/instrumentation-test, accessed 2026-03-04 (2026).

[19] Firebase, Start testing with the gcloud cli (firebase test lab), https://firebase.google.com/docs/test-lab/android/command-line, accessed 2026-03-04 (2026).

[20] N. R. Weeraddana, M. Alfadel, S. McIntosh, Characterizing timeout builds in continuous integration, IEEE Transactions on Software Engineering (2024). doi:10.1109/TSE.2024.3387840.

[21] J. Huang, et al., On the reruns of github actions workflows, Proceedings of the ACM on Software Engineering (2024). doi:10.1145/3795771.

[22] C. Panigutti, A. Perotti, A. Panisson, P. Bajardi, D. Pedreschi, Fairlens: Auditing black-box clinical decision support systems, Information Processing & Management 58 (5) (2021) 102657. doi:10.1016/j.ipm.2021.102657.

[23] P. Wang, H. Ding, The rationality of explanation or human capacity? understanding the impact of explainable artificial intelligence on human-AI trust and decision performance, Information Processing & Management 61 (4) (2024) 103732. doi:10.1016/j.ipm.2024.103732.

[24] A. M. Smith, D. S. Katz, K. E. Niemeyer, F. S. C. W. Group, Software citation principles, PeerJ Computer Science 2 (2016) e86. doi:10.7717/peerj-cs.86.

[25] C. S. Timperley, L. Herckis, C. Le Goues, M. Hilton, Understanding and improving artifact sharing in software engineering research, Empirical Software Engineering 26 (4) (2021). doi:10.1007/s10664-021-09973-5.

[26] M. Liu, X. Huang, W. He, Y. Xie, J. M. Zhang, X. Jing, Z. Chen, Y. Ma, Research artifacts in software engineering publications: Status and trends, Journal of Systems and Software 213 (2024) 112032. doi:10.1016/j.jss.2024.112032.

[27] GitHub Docs, Managing github actions settings for a repository (artifact and log retention), https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/enabling-features-for-your-repository/managing-github-actions-settings-for-a-repository, accessed: 2026-02-24 (2026).

[28] NIST, Fips pub 180-4: Secure hash standard (shs), https://csrc.nist.gov/publications/detail/fips/180/4/final, accessed 2026-02-25 (2015).

[29] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer, 2012. doi:10.1007/978-3-642-29044-2.

[30] W. J. Conover, Practical Nonparametric Statistics, 3rd Edition, Wiley, 1999.

[31] W. H. Kruskal, W. A. Wallis, Use of ranks in one-criterion variance analysis, Journal of the American Statistical Association 47 (260) (1952) 583–621. doi:10.1080/01621459.1952.10483441.

[32] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.

[33] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[34] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, Psychological Bulletin 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.

[35] M. Tomczak, E. Tomczak, The need to report effect size estimates revisited. an overview of some recommended measures of effect size, Trends in Sport Sciences 21 (1) (2014) 19–25.

[36] J. Romano, J. D. Kromrey, J. Coraggio, J. Skowronek, L. Devine, Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the nsse and other surveys?, in: Annual Meeting of the Florida Association of Institutional Research, 2006.

[37] J. W. Tukey, Exploratory Data Analysis, Addison-Wesley, Reading, MA, 1977.

[38] A. Agresti, Categorical Data Analysis, 3rd Edition, Wiley, 2013.

[39] R. G. Newcombe, Two-sided confidence intervals for the single proportion: comparison of seven methods, Statistics in Medicine 17 (8) (1998) 857–872. doi:10.1002/(SICI)1097-0258(19980430)17:8<857::AID-SIM777>3.0.CO;2-E.

[40] GitHub Docs, Dependency caching reference, https://docs.github.com/en/actions/reference/workflows-and-actions/dependency-caching, accessed 2026-03-04 (2026).

[41] L. Breiman, J. H. Friedman, R. A. Olshen, C. J. Stone, Classification and Regression Trees, Chapman and Hall/CRC, New York, 1984.

[42] C. Rudin, Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead, Nature Machine Intelligence 1 (5) (2019) 206–215. doi:10.1038/s42256-019-0048-x.

[43] P. Runeson, M. Höst, Guidelines for conducting and reporting case study research in software engineering, Empirical Software Engineering 14 (2) (2009) 131–164. doi:10.1007/s10664-008-9102-8.